

第 13 周：深度 RL 与策略优化

从函数逼近到自然梯度，策略空间中的优化艺术

优化算法课程讲义

摘要

上周我们学习了表格 RL 的收敛性理论。但当状态空间巨大（如图像输入）时，表格方法失效。本周我们进入**深度 RL**：用神经网络逼近值函数和策略，把 RL 问题转化为**参数空间中的优化问题**。我们将看到，策略梯度方法与 SGD、自然梯度与预条件优化、TRPO 与信任域方法之间的深刻联系——这正是优化理论在 RL 中的华丽应用。

目录

1 从表格到函数逼近：维度诅咒的破解	3
1.1 值函数逼近	3
1.2 DQN：深度 Q 网络	4
2 向量化环境与大规模并行	6
2.1 向量化环境	6
2.2 Isaac Gym：GPU 原生仿真	7
3 策略梯度：直接优化策略	8
3.1 策略优化问题	8
3.2 从轨迹出发：似然比技巧	9
3.3 策略梯度定理	10
3.4 具体策略的梯度计算	13
4 REINFORCE 与方差缩减	16
4.1 REINFORCE 算法	16
4.2 Baseline：方差缩减的艺术	16
4.3 GAE：广义优势估计	19
5 Actor-Critic：两个网络的协奏	21
5.1 Compatible Function Approximation	22
5.2 策略梯度的方差分析	23
5.3 On-policy vs Off-policy：A2C 与 Q-learning 的分野	25

6	Fisher 信息与自然梯度	26
6.1	Fisher 信息矩阵	27
6.2	自然梯度	29
7	TRPO 的理论基础：策略改进界	33
7.1	Performance Difference Lemma	33
7.2	重要性采样：用旧策略的数据估计新策略的期望	34
7.3	代理目标与近似界	35
8	TRPO：信任域策略优化	38
8.1	TRPO 的推导	38
8.2	TRPO 的求解	39
8.3	共轭梯度法求解 $F^{-1}g$	40
8.4	TRPO 的完整算法	42
9	算法对比与总结	43

1 从表格到函数逼近：维度诅咒的破解

回顾上周的局限

上周的 Q-learning 和值迭代都是**表格方法**：为每个 (s, a) 存储一个 Q 值。

问题：当状态空间巨大时，这不可行。

例 1.1 (维度诅咒——为什么需要函数逼近). **运筹学场景：车辆路径规划 (VRP)**

状态 = (当前位置, 已访问客户集合, 剩余容量, 时间)

- 100 个客户: $2^{100} \approx 10^{30}$ 种已访问子集
- 加上位置、容量、时间: 状态空间 $> 10^{35}$
- 存不下! 就算每个状态只用 1 bit, 也需要 10^{25} TB

Atari 游戏:

- 输入是 $84 \times 84 \times 4$ 的图像 (4 帧堆叠)
- 每个像素 256 个值: $256^{84 \times 84 \times 4} \approx 10^{68000}$ 种状态
- 比宇宙中的原子数 (10^{80}) 还多得多得多

解决方案: 不存储每个状态的值, 而是学一个**函数** $V_\theta(s)$ 来泛化。

1.1 值函数逼近

定义 1.1 (参数化值函数). 用参数 $\theta \in \mathbb{R}^d$ 表示值函数:

$$V_\theta(s) \approx V^*(s), \quad Q_\theta(s, a) \approx Q^*(s, a)$$

参数量 d 远小于状态数 $|S|$, 实现**压缩**和**泛化**。

函数逼近 = 监督学习 + 自举

目标: 找 θ 使得 $V_\theta(s)$ 接近真实 $V^*(s)$

问题: 我们不知道 $V^*(s)$!

解决: 用 **Bellman 目标**代替真实目标:

$$y_t = r_t + \gamma V_\theta(s_{t+1}) \quad (\text{自举: 用估计值构造目标})$$

损失函数:

$$L(\theta) = \mathbb{E} [(V_\theta(s_t) - y_t)^2]$$

这就是把 RL 问题转化为了**回归问题**!

例 1.2 (线性函数逼近——特征工程的艺术). **运筹学场景：库存管理**

状态 s = (库存水平, 在途订单, 需求预测, 星期几, 是否促销)

特征函数 $\phi(s) \in \mathbb{R}^{10}$:

- $\phi_1(s)$ = 库存水平
- $\phi_2(s)$ = 库存水平² (捕捉非线性)
- $\phi_3(s)$ = 在途订单
- $\phi_4(s)$ = 库存 $\times$ 是否促销 (交互项)
- ...

线性值函数：

$$V_{\theta}(s) = \theta^T \phi(s) = \sum_{i=1}^{10} \theta_i \phi_i(s)$$

优点：可解释！ $\theta_1 > 0$ 意味着库存高是好事（不会缺货）。

缺点：需要手工设计特征，表达能力有限。

1.2 DQN：深度 Q 网络

DQN 的革命性

2015 年，DeepMind 用 DQN 在 Atari 游戏上达到人类水平。

核心思想：用**深度神经网络**代替手工特征，让网络自己学特征！

定义 1.2 (DQN). 用神经网络 $Q_{\theta}(s, a)$ 逼近最优 Q 函数，损失函数：

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_{\theta}(s, a) - \underbrace{(r + \gamma \max_{a'} Q_{\theta-}(s', a'))}_{\text{TD 目标}} \right)^2 \right]$$

DQN 的两个关键技巧

1. Experience Replay (经验回放)

问题：连续的样本高度相关，违反 SGD 的 i.i.d. 假设。

解决：把经验 (s, a, r, s') 存入缓冲区 $\mathcal{D}$ ，随机采样训练。

运筹学类比：就像**历史订单数据库**。你不是只看最近一单，而是从历史中随机抽样学习，打破时间相关性。

三个易混点：(i) 采集与训练是两条交织的循环——agent 一直用 ϵ -greedy 在环境里走、把新 transition 灌进 $\mathcal{D}$ ，同时每步从 $\mathcal{D}$ 随机抽一个 **minibatch** (如 32 条，非全量) 做一次 SGD；经验回放复用的是已采集的数据，不替代采集。(ii) $\mathcal{D}$ 是**固定容量的 FIFO 滑动窗口** (如 10^6)，满了挤出最旧的，所以训练数据始终是”最近若干历史策略”采出来的混合。(iii) 能这样混用多个旧策略的数据，根源是 Q-learning 为 **off-policy**：目标 $r + \gamma \max_{a'} Q(s', a')$ 自包含，与”这条数据当初哪个策略采的”无关 (详见 §5.3)。换成 on-policy 方法 (A2C) 就不能直接用 replay。

2. Target Network (目标网络)

问题：TD 目标 $r + \gamma \max_{a'} Q_{\theta}(s', a')$ 依赖于 θ ，自己追自己，不稳定。

解决：另维护一个与 Q_θ 结构相同、参数滞后的目标网络 Q_{θ^-} 来算目标——即**同时维护两套参数**： θ （每步 SGD 更新）和 θ^- （慢更新的历史快照）。目标 $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$ 在求梯度时**当常数处理**（stop-gradient，不回传到 θ^- ）。 θ^- 的更新有两种：硬同步——每 C 步 $\theta^- \leftarrow \theta$ （Atari DQN $C \approx 10^4$ ）；软更新（Polyak）—— $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ ， τ 很小（如 0.005，DDPG/SAC 常用）。

优化类比：就像**延迟更新**。目标函数变化太快会导致优化震荡，固定一段时间让优化稳定。

Algorithm 1 DQN 算法

```

1: 初始化  $Q_\theta$  和目标网络  $Q_{\theta^-}$  ( $\theta^- \leftarrow \theta$ )，容量  $N$  的经验池  $\mathcal{D}$ 
2: for episode = 1, 2, ... do
3:   for step  $t = 1, 2, \dots$  do
4:     用  $\epsilon$ -greedy 选动作： $a_t = \begin{cases} \text{random} & \text{w.p. } \epsilon \\ \arg \max_a Q_\theta(s_t, a) & \text{otherwise} \end{cases}$ 
5:     执行  $a_t$ ，观察  $r_t, s_{t+1}$ 
6:     存入经验池（FIFO，满则弹出最旧）： $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$ 
7:     从  $\mathcal{D}$  随机采样 minibatch（大小  $B$ ，非全量）
8:     计算 TD 目标： $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$ 
9:     梯度下降： $\theta \leftarrow \theta - \alpha \nabla_\theta (Q_\theta(s, a) - y)^2$ 
10:    每  $C$  步： $\theta^- \leftarrow \theta$ 
11:   end for
12: end for
```

改进：Double DQN

原始 DQN 的目标里 $\max_{a'} Q_{\theta^-}(s', a')$ 同时用一个网络选动作又评估动作，会系统性**高估** Q 值。Double DQN 把两者拆开——用 online 网 Q_θ 选 $\arg \max$ ，用 target 网 Q_{θ^-} 打分：

$$y = r + \gamma Q_{\theta^-}(s', \arg \max_{a'} Q_\theta(s', a'))$$

几乎零额外成本，却显著降低高估偏差。基础 DQN 用上面的 $\max_{a'} Q_{\theta^-}$ 即可，二者择一。

例 1.3 (DQN 的运筹学应用——动态定价)。**场景：**航空公司动态定价，每个航班有 100 个座位。

状态：（剩余座位，距离起飞天数，竞争对手价格，历史需求）

动作：设定价格 $\in \{100, 150, 200, 250, 300, \dots\}$ （10 个离散价格）

奖励：当天售出座位 $\times$ 价格

DQN 的优势：

- 自动学习复杂的定价策略（比如“起飞前 3 天涨价”）
- 不需要建立需求模型（model-free）
- 可以处理高维状态（历史需求序列）

Experience Replay 在这里：从历史销售数据中学习，不是只看今天的销售。

2 向量化环境与大规模并行

深度 RL 的瓶颈

深度 RL 需要**海量样本**（通常 $10^6 \sim 10^9$ 次交互）。

瓶颈不是 GPU 计算，而是**环境仿真**——CPU 上一步步跑太慢了！

2.1 向量化环境

定义 2.1 (向量化环境). 同时运行 N 个独立环境副本，一次返回 N 个样本：

$$(s_1, \dots, s_N), (a_1, \dots, a_N) \rightarrow (r_1, \dots, r_N), (s'_1, \dots, s'_N)$$

例 2.1 (向量化环境——仓库调度并行仿真). **场景**：优化仓库机器人调度策略
传统方式（串行）：

- 仿真 1 个仓库，机器人走一步
- 策略网络前向传播，选动作
- 仿真下一步...
- 1000 步/秒

向量化（并行 1024 个仓库）：

- 同时仿真 1024 个仓库
- 策略网络一次处理 1024 个状态 (batch)
- GPU 并行，几乎同样时间
- 有效速度： $1000 \times 1024 = 1,024,000$ 步/秒

加速比：1000 倍！

EnvPool：高性能向量化

EnvPool 是目前最快的向量化环境库：

环境	Gym (1 env)	EnvPool (1024 envs)	加速比
CartPole	50K steps/s	3M steps/s	60×
Atari Pong	2K steps/s	1M steps/s	500×

关键技术：

- C++ 实现环境逻辑，避免 Python 开销
- 异步执行，隐藏延迟
- 内存预分配，零拷贝

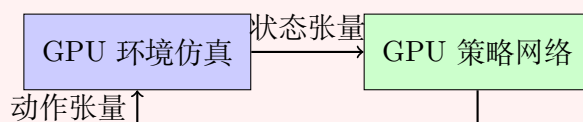
2.2 Isaac Gym: GPU 原生仿真

终极目标：全流程 GPU

EnvPool 仍然是 CPU 仿真 + GPU 训练。

Isaac Gym：环境仿真也在 GPU 上，实现**全流程 GPU**！

Isaac Gym 的架构



零 CPU-GPU 传输！

性能：

- 4096 个 Ant 机器人并行仿真
- RTX 3090 上：200,000+ steps/秒
- 5 分钟训练出能跑的机器人（传统方法需要几小时）

例 2.2 (Isaac Gym 应用——机器人步态优化). **问题**：让四足机器人学会走路

状态：关节角度、角速度、身体姿态 (~50 维)

动作：12 个关节的扭矩（连续控制）

奖励：前进速度 - 能耗 - 摔倒惩罚

传统训练：

- MuJoCo CPU 仿真，1 个机器人
- PPO 训练，需要 2000 万步
- 耗时：10 小时

Isaac Gym 训练：

- GPU 并行仿真，4096 个机器人
- 同样的 PPO 算法
- 耗时：**3 分钟**

加速比：200 倍！

并行化与优化的联系

大 batch SGD：向量化环境本质上是增大 batch size

回忆 SGD 方差： $\text{Var}[\nabla_{\text{batch}}] = \frac{\text{Var}[\nabla_{\text{single}}]}{B}$

更大的 batch:

- 梯度估计更准确 (方差降低 $\sqrt{B}$ 倍)
- 可以用更大的学习率
- 但不能无限大 (泛化可能变差)

RL 中的特殊性: 样本不是固定数据集, 而是策略生成的。并行环境提供了**多样性**, 相当于天然的数据增强。

3 策略梯度: 直接优化策略

两种 RL 范式

Value-based (DQN 等): 先学 Q^* , 再贪婪得到策略 $\pi(s) = \arg \max_a Q^*(s, a)$

Policy-based: 直接优化策略 π_θ !

为什么要直接优化策略?

- 连续动作空间: $\arg \max$ 难算
- 随机策略: 有时候随机更好 (博弈、探索)
- 更稳定: 策略小变化 $\rightarrow$ 行为小变化

3.1 策略优化问题

定义 3.1 (策略优化目标). 参数化策略 $\pi_\theta(a|s)$, 目标是最大化期望回报:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] = \mathbb{E}_{s_0} [V^{\pi_\theta}(s_0)]$$

其中 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 是轨迹。

策略优化 = 参数空间中的优化

这就是一个标准的优化问题!

决策变量: $\theta \in \mathbb{R}^d$ (网络参数)

目标函数: $J(\theta)$ (期望回报)

约束: 无 (或参数范围约束)

算法: 梯度上升 $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

挑战: $J(\theta)$ 没有解析形式, $\nabla_\theta J$ 怎么算?

例 3.1 (策略优化——物流网络设计). **问题:** 设计快递配送网络

策略 π_θ : 给定 (包裹信息, 当前网络状态), 输出 (路由决策的概率分布)

目标 $J(\theta)$: 期望配送时间 + 成本

难点:

- 配送时间依赖于整个网络的运行（长期效应）
- 需要仿真才能评估策略好坏
- ∇J 没有解析形式

策略梯度的威力：不需要环境模型，只需要能仿真！

3.2 从轨迹出发：似然比技巧

先别急着上定理

直接对 $J(\theta)$ 求导卡在哪？ $J(\theta) = \mathbb{E}_{\tau}[R(\tau)]$ ，期望里既有策略 π_{θ} ，又有我们不知道的环境动力学 P 。看起来要对 θ 求导就得知道 P 怎么随 θ 变——但其实不用。一个简单的恒等式就能把 P 彻底甩掉。

一条轨迹 $\tau = (s_0, a_0, s_1, a_1, \dots)$ 的概率为：

$$p_{\theta}(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

目标写成轨迹期望： $J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}}[R(\tau)]$ ，其中 $R(\tau) = \sum_t \gamma^t r_t$ 。

似然比技巧 (log-derivative trick)：核心是恒等式 $\nabla_{\theta} p_{\theta} = p_{\theta} \nabla_{\theta} \log p_{\theta}$ 。于是

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\theta}}[\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

这一步把“对概率分布求导”变成了“对样本求期望”——可以采样估计了。

动力学项自动消失

对 $\log p_{\theta}(\tau)$ 拆开：

$$\log p_{\theta}(\tau) = \underbrace{\log \rho_0(s_0)}_{\text{与}\theta\text{无关}} + \sum_t \log \pi_{\theta}(a_t | s_t) + \underbrace{\sum_t \log P(s_{t+1} | s_t, a_t)}_{\text{与}\theta\text{无关}}$$

求 ∇_{θ} 时，初始分布 ρ_0 和动力学 P 都不含 θ ，直接归零：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

这就是“不需要环境模型”的数学根源： P 在求导时被消掉了，我们只需要能仿真轨迹、再对策略的 $\log$ 概率求梯度。

代入得到**轨迹形式的策略梯度**：

定理 3.1 (策略梯度——轨迹形式)。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left(\sum_t \gamma^t r_t \right) \right]$$

这个形式可以直接采样估计：跑 N 条轨迹，对每条算 $\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 乘上整条轨迹的回报，再平均。这就是 REINFORCE 的雏形（稍后细讲）。

因果性：只看”未来”的奖励

上式中每个 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 都乘上了整条轨迹的回报，包括 t 时刻之前就已经发生的奖励。但 t 时刻的动作不可能影响过去的奖励，所以对 $t' < t$ 有

$$\mathbb{E}_{\tau} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot r_{t'}] = 0$$

(给定 s_t 之前的历史， $r_{t'}$ 与 a_t 条件独立，而 score 的条件期望为零)。去掉这些零项，回报缩成 **reward-to-go**：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right], \quad G_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$$

(注意保留的奖励是 $\sum_{t' \geq t} \gamma^{t'} r_{t'} = \gamma^t G_t$ ，前面那个 γ^t 不能丢——它正是下一节折扣状态分布 d^{π} 的来源；实践中常被省略，导致有偏的”无折扣状态加权”版本。) 这一步不改期望值，但**大幅降方差**——后面讲 baseline 时会反复用到这个思路。

通往下一节的紧凑形式

对 reward-to-go 再取一次条件期望， $\mathbb{E}[G_t | s_t, a_t] = Q^{\pi_{\theta}}(s_t, a_t)$ ，就得到逐步形式

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t) \right]$$

最后把”跨轨迹的时间求和 $\sum_t \gamma^t$ ”按状态归并成**折扣状态访问分布** $d^{\pi}(s) = \sum_t \gamma^t P(s_t = s)$ ，时间下标 t 就消失了，落到下一节 Sutton 定理的紧凑写法 $\mathbb{E}_{s \sim d^{\pi}, a \sim \pi_{\theta}}[\dots]$ 。下一节就顺这条路给出主证明（经典的 Bellman 递归证明作为对照附在其后）。

3.3 策略梯度定理

定理 3.2 (策略梯度定理 - Sutton et al., 1999).

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot Q^{\pi_{\theta}}(s, a)]$$

这个公式在说什么

$\nabla_{\theta} \log \pi_{\theta}(a | s)$ ：如果我调整 θ ，动作 a 的概率会怎么变？

$Q^{\pi_{\theta}}(s, a)$ ：动作 a 有多好？

乘起来：好的动作，增加其概率；坏的动作，减少其概率。

这就是**强化**的本质！

从轨迹形式顺推。承接上一节的轨迹形式（已用 reward-to-go 整理，每个时刻带 γ^t 因子）：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right], \quad G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$$

Step 1 (条件期望): 对每个时刻用塔性质, 给定 (s_t, a_t) 后 $\mathbb{E}[G_t \mid s_t, a_t] = Q^{\pi_\theta}(s_t, a_t)$:

$$\nabla_\theta J = \mathbb{E}_\tau \left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^{\pi_\theta}(s_t, a_t) \right]$$

Step 2 (拆成各时刻边缘): 因 $\gamma < 1$ 使各项绝对可和, 由 Fubini 定理交换求和与期望; 记 $p_t(s) = P(s_t = s \mid \pi_\theta)$ 为 t 时刻的状态边缘分布:

$$= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_t, a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^{\pi_\theta}(s_t, a_t)]$$

Step 3 (按状态归并, γ^t 被吸收): 把动作期望用 log-trick 写成 $\sum_a \pi_\theta \nabla_\theta \log \pi_\theta$, 并交换 $\sum_t$ 与对状态的求和 ($p_t(s) \geq 0$, Tonelli), 按状态 s 归并:

$$= \sum_s \underbrace{\left(\sum_{t=0}^{\infty} \gamma^t p_t(s) \right)}_{= d^\pi(s)} \sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a)$$

括号里的 $d^\pi(s) = \sum_{t \geq 0} \gamma^t P(s_t = s \mid \pi_\theta)$ 正是**折扣状态访问分布**——轨迹形式里那个跨时刻的 $\sum_t \gamma^t$, 归并后就成了它。

Step 4 (写回期望):

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a)]$$

即定理所述。注意紧凑写法里的 $\mathbb{E}_{\pi_\theta}$, 状态严格按 d^π 取——常被误读为“少了对时间的求和”, 其实那个求和已被打包进 d^π 。□

另一种证明: Bellman 递归展开 (Sutton et al. 1999 原始路径)。不借助轨迹形式, 直接从 $\nabla_\theta V^\pi$ 的递归出发也能得到同样结果, 分步展开如下。

Step 1: 定义并展开 $J(\theta)$

设起始状态分布为 $\rho_0(s)$, 则:

$$J(\theta) = \sum_{s_0} \rho_0(s_0) V^{\pi_\theta}(s_0)$$

其中值函数可以写成:

$$V^\pi(s) = \sum_a \pi_\theta(a \mid s) Q^\pi(s, a)$$

Step 2: 对 θ 求 $\nabla V^\pi(s)$

$$\begin{aligned} \nabla_\theta V^\pi(s) &= \nabla_\theta \left[\sum_a \pi_\theta(a \mid s) Q^\pi(s, a) \right] \\ &= \sum_a [\nabla_\theta \pi_\theta(a \mid s) \cdot Q^\pi(s, a) + \pi_\theta(a \mid s) \cdot \nabla_\theta Q^\pi(s, a)] \end{aligned}$$

这里用了乘法法则: $\nabla(fg) = (\nabla f)g + f(\nabla g)$

Step 3: 展开 $\nabla_\theta Q^\pi(s, a)$

由 Bellman 方程：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$$

由于 R 和 P 不依赖于 θ ：

$$\nabla_\theta Q^\pi(s, a) = \gamma \sum_{s'} P(s'|s, a) \nabla_\theta V^\pi(s')$$

Step 4: 递归展开

将 Step 3 代入 Step 2:

$$\begin{aligned} \nabla_\theta V^\pi(s) &= \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) + \sum_a \pi_\theta(a|s) \gamma \sum_{s'} P(s'|s, a) \nabla_\theta V^\pi(s') \\ &= \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) + \gamma \sum_{s'} P^\pi(s'|s) \nabla_\theta V^\pi(s') \end{aligned}$$

其中 $P^\pi(s'|s) = \sum_a \pi_\theta(a|s) P(s'|s, a)$ 是策略下的转移概率。

这是一个**递归方程**！继续展开：

$$\begin{aligned} \nabla_\theta V^\pi(s) &= \sum_a \nabla_\theta \pi_\theta(a|s) Q + \gamma \sum_{s'} P^\pi(s'|s) \left[\sum_a \nabla_\theta \pi_\theta(a|s') Q + \gamma \sum_{s''} P^\pi(s''|s') \nabla_\theta V^\pi(s'') \right] \\ &= \sum_a \nabla_\theta \pi_\theta(a|s) Q + \gamma \sum_{s'} P^\pi(s'|s) \sum_a \nabla_\theta \pi_\theta(a|s') Q + \gamma^2 \sum_{s''} P^\pi(s''|s, 2\text{步}) \nabla_\theta V^\pi(s'') \end{aligned}$$

Step 5: 用状态访问分布整理

定义**折扣状态访问分布**：

$$d^\pi(s'|s_0) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s' | s_0, \pi)$$

这是从 s_0 出发，在策略 π 下，访问状态 s' 的折扣概率。

将无穷展开整理：

$$\begin{aligned} \nabla_\theta V^\pi(s_0) &= \sum_{t=0}^{\infty} \gamma^t \sum_s P(s_t = s | s_0, \pi) \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) \\ &= \sum_s d^\pi(s|s_0) \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) \end{aligned}$$

因此：

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s_0} \rho_0(s_0) \nabla_\theta V^\pi(s_0) \\ &= \sum_{s_0} \rho_0(s_0) \sum_s d^\pi(s|s_0) \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) \\ &= \sum_s d^\pi(s) \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s, a) \end{aligned}$$

其中 $d^\pi(s) = \sum_{s_0} \rho_0(s_0) d^\pi(s|s_0)$ 是整体状态访问分布。

Step 6: Log-trick

关键恒等式： $\nabla_\theta \pi_\theta(a|s) = \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)$

代入：

$$\begin{aligned}\nabla_{\theta} J &= \sum_s d^{\pi}(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a) \\ &= \mathbb{E}_{s \sim d^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)]\end{aligned}$$

□

Log-trick 的验证

验证 $\nabla \pi = \pi \nabla \log \pi$ ：

设 $\pi_{\theta}(a|s) = f(\theta)$ ，则 $\log \pi = \log f(\theta)$

$$\nabla \log \pi = \frac{\nabla f}{f} = \frac{\nabla \pi}{\pi}$$

两边乘 π ： $\pi \nabla \log \pi = \nabla \pi$ ✓

Log-trick 的深意

为什么用 $\nabla \log \pi$ 而不是 $\nabla \pi / \pi$ ？

数值稳定：当 π 接近 0 时， $\nabla \pi / \pi$ 爆炸，但 $\nabla \log \pi$ 有界。

几何意义： $\nabla \log \pi$ 是 **Fisher score**，在概率空间中有特殊地位（后面讲自然梯度会用到）。

采样友好： $\mathbb{E}_{\pi}[f \cdot \nabla \log \pi] = \mathbb{E}_{\pi}[f \cdot \frac{\nabla \pi}{\pi}] = \sum_a f(a) \nabla \pi(a)$

无论 f 是什么，都可以用采样估计！

例 3.2 (策略梯度的直觉——调整轮盘赌)。想象你在玩轮盘赌，可以调整每个数字的概率。

策略 π_{θ} ：数字 1-10 的概率分布

奖励：押中 5 得 100 分，押中其他得 0 分

一次试验：按 π_{θ} 采样，假设采到了 7，奖励 0。

策略梯度更新：

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(7) \cdot 0 = \theta$$

奖励是 0，不更新。

另一次试验：采到了 5，奖励 100。

更新：

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(5) \cdot 100$$

$\nabla_{\theta} \log \pi_{\theta}(5)$ 指向“增加 5 的概率”的方向，乘以 100（大奖励），大幅调整。

多次试验后：5 的概率越来越高！

3.4 具体策略的梯度计算

Softmax 策略的梯度

设置：离散动作 $a \in \{1, 2, \dots, K\}$ ，状态特征 $\phi(s) \in \mathbb{R}^d$

Softmax 策略：

$$\pi_{\theta}(a|s) = \frac{\exp(\theta_a^T \phi(s))}{\sum_{b=1}^K \exp(\theta_b^T \phi(s))}$$

其中 $\theta = (\theta_1, \dots, \theta_K) \in \mathbb{R}^{K \times d}$ 。

计算 $\log \pi$ ：

$$\log \pi_{\theta}(a|s) = \theta_a^T \phi(s) - \log \sum_{b=1}^K \exp(\theta_b^T \phi(s))$$

对 θ_a 求梯度：

$$\begin{aligned} \nabla_{\theta_a} \log \pi_{\theta}(a|s) &= \phi(s) - \frac{\exp(\theta_a^T \phi(s))}{\sum_b \exp(\theta_b^T \phi(s))} \phi(s) \\ &= \phi(s) - \pi_{\theta}(a|s) \phi(s) \\ &= (1 - \pi_{\theta}(a|s)) \phi(s) \end{aligned}$$

对 θ_b ($b \neq a$) 求梯度：

$$\nabla_{\theta_b} \log \pi_{\theta}(a|s) = -\pi_{\theta}(b|s) \phi(s)$$

整合成向量形式：

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = (\mathbf{e}_a - \pi_{\theta}(\cdot|s)) \otimes \phi(s)$$

其中 $\mathbf{e}_a$ 是第 a 个标准基向量， $\otimes$ 是外积。

数值例子： $K = 3$ 动作， $\phi(s) = (1, 2)^T$ ， $\theta = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$

计算 π ：

$$\theta_1^T \phi = 1 \cdot 1 + 0 \cdot 2 = 1$$

$$\theta_2^T \phi = 0 \cdot 1 + 1 \cdot 2 = 2$$

$$\theta_3^T \phi = 0 \cdot 1 + 0 \cdot 2 = 0$$

$$\pi(1|s) = \frac{e^1}{e^1 + e^2 + e^0} = \frac{2.72}{2.72 + 7.39 + 1} = 0.244$$

$$\pi(2|s) = \frac{e^2}{e^1 + e^2 + e^0} = \frac{7.39}{11.11} = 0.665$$

$$\pi(3|s) = \frac{e^0}{e^1 + e^2 + e^0} = \frac{1}{11.11} = 0.090$$

若采样到 $a = 2$ ， $\nabla_{\theta_2} \log \pi(2|s) = (1 - 0.665) \cdot (1, 2)^T = (0.335, 0.670)^T$

高斯策略的梯度——连续动作

设置：连续动作 $a \in \mathbb{R}$ ，均值由网络输出

高斯策略：

$$\pi_{\theta}(a|s) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(a - \mu_{\theta}(s))^2}{2\sigma^2}\right)$$

其中 $\mu_{\theta}(s) = \theta^T \phi(s)$ 是线性均值函数， σ 固定。

计算 $\log \pi$ ：

$$\log \pi_{\theta}(a|s) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(a - \mu_{\theta}(s))^2}{2\sigma^2}$$

对 θ 求梯度:

$$\begin{aligned}\nabla_{\theta} \log \pi_{\theta}(a|s) &= -\frac{2(a - \mu_{\theta}(s))(-\nabla_{\theta} \mu_{\theta}(s))}{2\sigma^2} \\ &= \frac{(a - \mu_{\theta}(s))}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s) \\ &= \frac{(a - \mu_{\theta}(s))}{\sigma^2} \phi(s)\end{aligned}$$

直觉:

- 如果 $a > \mu$ (动作比均值大) 且获得正奖励, 梯度方向增大 μ
- 如果 $a < \mu$ (动作比均值小) 且获得正奖励, 梯度方向减小 μ
- σ^2 越大, 梯度越小 (不确定性大时更新谨慎)

数值例子: $\phi(s) = (1, 1)^T$, $\theta = (2, 1)^T$, $\sigma = 0.5$

$$\mu = \theta^T \phi = 2 \cdot 1 + 1 \cdot 1 = 3$$

采样 $a = 3.5$ (比均值大 0.5), 奖励 $r = 10$

$$\nabla_{\theta} \log \pi = \frac{3.5-3}{0.25}(1, 1)^T = 2 \cdot (1, 1)^T = (2, 2)^T$$

$$\text{策略梯度: } \nabla_{\theta} J \approx (2, 2)^T \cdot 10 = (20, 20)^T$$

更新后 μ 增大, 使得更大的动作更可能被选中。

多维高斯策略——对角协方差

设置: $a \in \mathbb{R}^m$, 对角协方差 $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_m^2)$

策略:

$$\pi_{\theta}(a|s) = \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{(a_i - \mu_{\theta,i}(s))^2}{2\sigma_i^2}\right)$$

Log-likelihood:

$$\log \pi = -\frac{m}{2} \log(2\pi) - \sum_{i=1}^m \log \sigma_i - \sum_{i=1}^m \frac{(a_i - \mu_{\theta,i})^2}{2\sigma_i^2}$$

对均值参数的梯度:

$$\nabla_{\theta} \log \pi = \sum_{i=1}^m \frac{a_i - \mu_{\theta,i}}{\sigma_i^2} \nabla_{\theta} \mu_{\theta,i}$$

如果 σ_i 也是可学习参数 (参数化为 $\log \sigma_i$ 避免负数):

$$\frac{\partial \log \pi}{\partial \log \sigma_i} = \frac{(a_i - \mu_i)^2}{\sigma_i^2} - 1$$

直觉: 如果 $(a_i - \mu_i)^2 > \sigma_i^2$ (偏离较大), 增大 σ_i ; 否则减小。

4 REINFORCE 与方差缩减

4.1 REINFORCE 算法

Algorithm 2 REINFORCE

```
1: 初始化策略网络  $\pi_\theta$ 
2: for episode = 1, 2, ... do
3:   采样轨迹  $\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$ 
4:   计算回报  $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ 
5:   梯度估计:  $\hat{g} = \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$ 
6:   更新:  $\theta \leftarrow \theta + \alpha \hat{g}$ 
7: end for
```

REINFORCE 的问题

REINFORCE 是**无偏的**: $\mathbb{E}[\hat{g}] = \nabla J(\theta)$

但**方差巨大**!

原因: G_t 可能是 +1000 也可能是 -500, 波动剧烈。

后果: 需要大量样本, 收敛慢, 训练不稳定。

4.2 Baseline: 方差缩减的艺术

定理 4.1 (Baseline 不改变期望). 对任意只依赖于状态的函数 $b(s)$:

$$\mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] = 0$$

因此可以用 $Q(s, a) - b(s)$ 代替 $Q(s, a)$, 期望不变, 但方差可能大大减少!

证明.

$$\begin{aligned}\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] &= b(s) \sum_a \pi_\theta(a|s) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\ &= b(s) \sum_a \nabla_\theta \pi_\theta(a|s) \\ &= b(s) \underbrace{\nabla_\theta \sum_a \pi_\theta(a|s)}_{=1} = 0\end{aligned}$$

□

定理 4.2 (最优 Baseline 的精确形式). 使方差最小的 baseline 为:

$$b^*(s) = \frac{\mathbb{E}_{a \sim \pi} [Q(s, a) \|\nabla_\theta \log \pi(a|s)\|^2]}{\mathbb{E}_{a \sim \pi} [\|\nabla_\theta \log \pi(a|s)\|^2]}$$

证明. **Step 1**: 写出方差

设 $g(s, a) = \nabla_\theta \log \pi(a|s)$, 原始梯度估计 $\hat{g} = g(s, a) \cdot Q(s, a)$ 。

带 baseline 的估计: $\hat{g}_b = g(s, a) \cdot (Q(s, a) - b(s))$

方差（对于单个状态 s ，考虑标量情况简化）：

$$\begin{aligned}\text{Var}_a[\hat{g}_b] &= \mathbb{E}_a[(g \cdot (Q - b))^2] - (\mathbb{E}_a[g \cdot (Q - b)])^2 \\ &= \mathbb{E}_a[g^2(Q - b)^2] - (\mathbb{E}_a[gQ] - \underbrace{b \mathbb{E}_a[g]}_{=0})^2 \\ &= \mathbb{E}_a[g^2(Q - b)^2] - (\mathbb{E}_a[gQ])^2\end{aligned}$$

Step 2: 对 b 求导

只有第一项依赖于 b ：

$$\begin{aligned}\frac{\partial}{\partial b} \mathbb{E}_a[g^2(Q - b)^2] &= \mathbb{E}_a[g^2 \cdot 2(Q - b) \cdot (-1)] \\ &= -2\mathbb{E}_a[g^2Q] + 2b\mathbb{E}_a[g^2]\end{aligned}$$

Step 3: 令导数为零

$$-2\mathbb{E}_a[g^2Q] + 2b^*\mathbb{E}_a[g^2] = 0$$

$$b^*(s) = \frac{\mathbb{E}_a[g^2Q]}{\mathbb{E}_a[g^2]} = \frac{\mathbb{E}_a[\|\nabla \log \pi\|^2 Q]}{\mathbb{E}_a[\|\nabla \log \pi\|^2]}$$

Step 4: 验证是最小值

$$\frac{\partial^2}{\partial b^2} \text{Var} = 2\mathbb{E}_a[g^2] > 0, \text{ 确实是最小值。}$$

□

最优 Baseline 的近似

精确的 $b^*(s)$ 需要知道 $\|\nabla \log \pi\|^2$ 的分布，实际中用近似：

近似 1: 假设 $\|\nabla \log \pi\|^2$ 对所有 a 近似相同

$$b^*(s) \approx \frac{\mathbb{E}_a[Q(s, a)]}{\mathbb{E}_a[1]} = \mathbb{E}_a[Q(s, a)] = V(s)$$

近似 2: 用状态值函数网络 $V_\phi(s)$ 估计 $b^*(s) \approx V(s)$

这就是为什么 Actor-Critic 中的 Critic 学习 V 而不是 Q ！

数值验证: 假设 3 个动作， $Q = (10, 20, 30)$ ， $\|\nabla \log \pi\|^2 = (1, 2, 1)$ （向量范数）

$$\text{精确 } b^* = \frac{10 \cdot 1 + 20 \cdot 2 + 30 \cdot 1}{1 + 2 + 1} = \frac{80}{4} = 20$$

$$\text{近似 } b = V = \frac{10 + 20 + 30}{3} = 20 \text{（假设均匀策略）}$$

这里两者相同！但一般情况下会有差异。

Advantage 函数与方差缩减

定义优势函数 $A(s, a) = Q(s, a) - V(s)$

性质 1: $\mathbb{E}_{a \sim \pi}[A(s, a)] = 0$

证明: $\mathbb{E}_a[A] = \mathbb{E}_a[Q - V] = V - V = 0$

性质 2: $A(s, a)$ 衡量动作 a 相对于平均的好坏

- $A > 0$: 动作比平均好，应该增加概率
- $A < 0$: 动作比平均差，应该减少概率

- $A = 0$: 动作和平均一样

方差分析:

设 Q 的均值 $\mu = V$, 方差 σ_Q^2

无 baseline: 信号 $Q \in [\mu - 2\sigma_Q, \mu + 2\sigma_Q]$

有 baseline: 信号 $A = Q - V \in [-2\sigma_Q, 2\sigma_Q]$

如果 $\mu \gg \sigma_Q$ (比如物流成本总是正的), 无 baseline 的信号绝对值大但信噪比低。

例 4.1 (方差缩减的效果——物流调度). **场景**: 每天调度 100 辆车

奖励 (无 baseline):

- 好调度: 总成本-10000 (奖励 +10000)
- 差调度: 总成本-15000 (奖励 +5000)
- 信号: +10000 vs +5000, 差异 5000
- 但数值本身很大, 方差大

奖励 (有 baseline, $b = 12500$):

- 好调度: $10000 - 12500 = -2500$
- 差调度: $5000 - 12500 = -7500$
- 信号: -2500 vs -7500, 差异还是 5000
- 但数值小了, 方差小!

最优 baseline $V(s)$: 让优势 $A(s, a)$ 在 0 附近波动, 方差最小。

方差缩减的定量分析

设 G 是回报, 期望 μ , 方差 σ^2 。设 $\psi = \nabla \log \pi$ 。

无 baseline 的梯度方差:

$$\begin{aligned}\text{Var}[\psi G] &= \mathbb{E}[(\psi G)^2] - (\mathbb{E}[\psi G])^2 \\ &= \mathbb{E}[\|\psi\|^2 G^2] - \|\mathbb{E}[\psi G]\|^2 \\ &= \mathbb{E}[\|\psi\|^2] \mathbb{E}[G^2] - \|\mathbb{E}[\psi G]\|^2 \quad (\text{若 } \psi \text{ 和 } G \text{ 独立}) \\ &= \mathbb{E}[\|\psi\|^2] (\sigma^2 + \mu^2) - \|\nabla J\|^2\end{aligned}$$

有 baseline 的梯度方差:

$$\begin{aligned}\text{Var}[\psi(G - b)] &= \mathbb{E}[\|\psi\|^2 (G - b)^2] - \|\mathbb{E}[\psi(G - b)]\|^2 \\ &= \mathbb{E}[\|\psi\|^2] (\sigma^2 + (\mu - b)^2) - \|\nabla J\|^2\end{aligned}$$

方差减少量:

$$\Delta \text{Var} = \mathbb{E}[\|\psi\|^2] [\mu^2 - (\mu - b)^2] = \mathbb{E}[\|\psi\|^2] \cdot b(2\mu - b)$$

当 $b = \mu = V$ 时: $\Delta \text{Var} = \mathbb{E}[\|\psi\|^2] \cdot \mu^2$

数值例子: $\mu = 10000, \sigma = 1000, b = V = 10000$

无 baseline: $\text{Var} \propto 10^6 + 10^8 = 1.01 \times 10^8$

有 baseline: $\text{Var} \propto 10^6 + 0 = 10^6$

方差降低 101 倍!

4.3 GAE: 广义优势估计

TD 估计 vs MC 估计

TD(0): $\hat{A}^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t$ (1 步 TD 误差)

- 优点: 低方差 (只用一步)
- 缺点: 有偏差 (V 不准时)

MC: $\hat{A}^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t) = G_t - V(s_t)$

- 优点: 无偏
- 缺点: 高方差 (用整条轨迹)

GAE: 在两者之间平滑过渡!

定义 4.1 (n 步优势估计).

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t) = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$$

推导 $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$.

$$\begin{aligned} \hat{A}_t^{(n)} &= r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n}) - V(s_t) \\ &= [r_t + \gamma V(s_{t+1}) - V(s_t)] \\ &\quad + \gamma [r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1})] \\ &\quad + \dots \\ &\quad + \gamma^{n-1} [r_{t+n-1} + \gamma V(s_{t+n}) - V(s_{t+n-1})] \\ &= \delta_t + \gamma \delta_{t+1} + \dots + \gamma^{n-1} \delta_{t+n-1} \\ &= \sum_{l=0}^{n-1} \gamma^l \delta_{t+l} \end{aligned}$$

□

定义 4.2 ($\text{GAE}(\lambda)$).

$$\hat{A}_t^{\text{GAE}(\lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}$$

是所有 n 步估计的指数加权平均。

定理 4.3 (GAE 的简洁形式).

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

证明.

$$\begin{aligned} \hat{A}_t^{\text{GAE}} &= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} \\ &= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \delta_{t+l} \end{aligned}$$

交换求和顺序。对于固定的 l , δ_{t+l} 出现在 $n > l$ 的所有项中:

$$\begin{aligned} \hat{A}_t^{\text{GAE}} &= (1 - \lambda) \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} \sum_{n=l+1}^{\infty} \lambda^{n-1} \\ &= (1 - \lambda) \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} \cdot \frac{\lambda^l}{1 - \lambda} \\ &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \end{aligned}$$

其中用了几何级数: $\sum_{n=l+1}^{\infty} \lambda^{n-1} = \lambda^l + \lambda^{l+1} + \dots = \frac{\lambda^l}{1-\lambda}$

□

GAE 的数值计算

实际中, 轨迹有限长 T , 从后往前计算:

$$\hat{A}_T^{\text{GAE}} = 0 \text{ (终止状态)}$$

$$\hat{A}_{T-1}^{\text{GAE}} = \delta_{T-1}$$

$$\hat{A}_{T-2}^{\text{GAE}} = \delta_{T-2} + \gamma\lambda\hat{A}_{T-1}^{\text{GAE}}$$

...

$$\hat{A}_t^{\text{GAE}} = \delta_t + \gamma\lambda\hat{A}_{t+1}^{\text{GAE}}$$

这是一个一次遍历的 $O(T)$ 算法!

数值例子: $\gamma = 0.99$, $\lambda = 0.95$, 轨迹 $(r_0, r_1, r_2) = (1, 2, 3)$

假设 $V(s_0) = 10, V(s_1) = 11, V(s_2) = 12, V(s_3) = 0$ (终止)

$$\delta_0 = 1 + 0.99 \times 11 - 10 = 1.89$$

$$\delta_1 = 2 + 0.99 \times 12 - 11 = 2.88$$

$$\delta_2 = 3 + 0.99 \times 0 - 12 = -9$$

GAE (从后往前):

$$\hat{A}_2 = -9$$

$$\hat{A}_1 = 2.88 + 0.99 \times 0.95 \times (-9) = 2.88 - 8.46 = -5.58$$

$$\hat{A}_0 = 1.89 + 0.99 \times 0.95 \times (-5.58) = 1.89 - 5.25 = -3.36$$

λ 的选择

λ	偏差	方差	等价于
0	高 (依赖 V)	低	TD(0): $\hat{A} = \delta_t$
1	低 (无偏)	高	MC: $\hat{A} = G_t - V(s_t)$
0.95	中等	中等	实践中常用

偏差-方差权衡:

- λ 小: 更信任 V 的估计, 偏差大但方差小
- λ 大: 更信任实际回报, 偏差小但方差大

5 Actor-Critic: 两个网络的协奏

REINFORCE 的另一个问题

REINFORCE 用 G_t (整个轨迹的回报) 作为 Q 的估计。

问题: 必须等到 episode 结束才能更新!

解决: 用一个神经网络来估计 Q 或 V , 实时更新。

这就是 Actor-Critic:

- **Actor** (演员): 策略网络 π_θ , 选动作
- **Critic** (评委): 值网络 V_ϕ , 评价好坏

定义 5.1 (Actor-Critic). **Critic 更新** (TD 学习):

$$\phi \leftarrow \phi - \beta \nabla_\phi (V_\phi(s) - (r + \gamma V_\phi(s')))^2$$

Actor 更新 (策略梯度):

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) \cdot \underbrace{(r + \gamma V_\phi(s') - V_\phi(s))}_{\text{TD 误差} \approx A(s,a)}$$

定理 5.1 (TD 误差是优势函数的无偏估计). 设 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, 其中 $V = V^\pi$ 是真实值函数。则:

$$\mathbb{E}[\delta_t | s_t, a_t] = A^\pi(s_t, a_t)$$

证明.

$$\begin{aligned} \mathbb{E}[\delta_t | s_t, a_t] &= \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) | s_t, a_t] - V^\pi(s_t) \\ &= R(s_t, a_t) + \gamma \sum_{s'} P(s' | s_t, a_t) V^\pi(s') - V^\pi(s_t) \\ &= Q^\pi(s_t, a_t) - V^\pi(s_t) \quad (\text{由 } Q \text{ 的定义}) \\ &= A^\pi(s_t, a_t) \end{aligned}$$

□

推论 5.1 (TD 误差的期望为零).

$$\mathbb{E}_{a_t \sim \pi}[\delta_t | s_t] = \mathbb{E}_{a_t \sim \pi}[A^\pi(s_t, a_t) | s_t] = 0$$

证明.

$$\begin{aligned} \mathbb{E}_{a \sim \pi}[A^\pi(s, a)] &= \mathbb{E}_{a \sim \pi}[Q^\pi(s, a) - V^\pi(s)] \\ &= \mathbb{E}_{a \sim \pi}[Q^\pi(s, a)] - V^\pi(s) \\ &= V^\pi(s) - V^\pi(s) = 0 \end{aligned}$$

□

5.1 Compatible Function Approximation

问题

当用函数逼近 $\hat{A}_w(s, a)$ 代替真实 $A^\pi(s, a)$ 时, 策略梯度还对吗?
 一般情况下, 用近似 $\hat{A}$ 会引入**偏差**.
 但有一个特殊情况: 如果 $\hat{A}$ 满足”兼容性条件”, 偏差为零!

定理 5.2 (Compatible Function Approximation - Sutton et al., 1999). 若 $\hat{A}_w(s, a)$ 满足:

1. **兼容性**: $\nabla_a \hat{A}_w(s, a) = \nabla_\theta \log \pi_\theta(a | s)$
2. **最小二乘**: $w = \arg \min_w \mathbb{E}_{s, a \sim \pi}[(A^\pi(s, a) - \hat{A}_w(s, a))^2]$

则用 $\hat{A}_w$ 代替 A^π 不影响策略梯度:

$$\nabla_\theta J = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a | s) \cdot \hat{A}_w(s, a)]$$

证明. **Step 1**: 兼容性条件的形式

条件 1 意味着 $\hat{A}_w(s, a) = w^T \nabla_\theta \log \pi_\theta(a | s) + b(s)$

其中 $b(s)$ 是只依赖于 s 的基线。

Step 2: 最小二乘条件的一阶条件

$$\nabla_w \mathbb{E}[(A - \hat{A}_w)^2] = 0$$

$$\mathbb{E}[(A^\pi - \hat{A}_w) \nabla_w \hat{A}_w] = 0$$

$$\mathbb{E}[(A^\pi - \hat{A}_w) \nabla_\theta \log \pi] = 0 \quad (\text{由条件 1})$$

Step 3: 策略梯度的差

$$\begin{aligned} &\mathbb{E}[\nabla_\theta \log \pi \cdot \hat{A}_w] - \mathbb{E}[\nabla_\theta \log \pi \cdot A^\pi] \\ &= \mathbb{E}[\nabla_\theta \log \pi \cdot (\hat{A}_w - A^\pi)] \\ &= -\mathbb{E}[(A^\pi - \hat{A}_w) \nabla_\theta \log \pi] \\ &= 0 \quad (\text{由 Step 2}) \end{aligned}$$

因此 $\mathbb{E}[\nabla_\theta \log \pi \cdot \hat{A}_w] = \mathbb{E}[\nabla_\theta \log \pi \cdot A^\pi] = \nabla_\theta J$.

□

例 5.1 (Compatible Function Approximation 的实例). 对于高斯策略 $\pi_\theta(a|s) = \mathcal{N}(\theta^T \phi(s), \sigma^2)$:

$$\nabla_\theta \log \pi = \frac{a - \theta^T \phi(s)}{\sigma^2} \phi(s)$$

$$\text{兼容的优势函数形式: } \hat{A}_w(s, a) = w^T \cdot \frac{a - \theta^T \phi(s)}{\sigma^2} \phi(s)$$

这是一个关于 $(a - \mu)$ 的线性函数!

直觉: 兼容函数只需要建模”动作偏离均值时优势如何变化”, 不需要建模完整的 Q 函数。

5.2 策略梯度的方差分析

定理 5.3 (策略梯度估计的方差). 设 $\hat{g} = \nabla_\theta \log \pi_\theta(a|s) \cdot \hat{A}(s, a)$ 是策略梯度的单样本估计, 则:

$$\text{Var}[\hat{g}] = \mathbb{E}[\|\nabla \log \pi\|^2 \hat{A}^2] - \|\nabla J\|^2$$

证明. 设 $\psi = \nabla_\theta \log \pi$, 则 $\hat{g} = \psi \cdot \hat{A}$ 。

$$\text{方差定义: } \text{Var}[\hat{g}] = \mathbb{E}[\hat{g}\hat{g}^T] - \mathbb{E}[\hat{g}]\mathbb{E}[\hat{g}]^T$$

$$\mathbb{E}[\hat{g}] = \nabla J \quad (\text{无偏性})$$

$$\mathbb{E}[\hat{g}\hat{g}^T] = \mathbb{E}[\psi\psi^T \hat{A}^2]$$

对于标量情况 (θ 是一维):

$$\text{Var}[\hat{g}] = \mathbb{E}[\psi^2 \hat{A}^2] - (\nabla J)^2$$

对于多维情况, 迹 (总方差):

$$\text{tr}(\text{Var}[\hat{g}]) = \mathbb{E}[\|\psi\|^2 \hat{A}^2] - \|\nabla J\|^2$$

□

方差的数量级估计

假设:

- $\|\nabla \log \pi\| \sim O(1)$ (标准化)
- $\hat{A} \sim O(R_{\max}/(1 - \gamma))$ (优势函数数量级)
- $\|\nabla J\| \sim O(R_{\max}/(1 - \gamma))$

$$\text{方差} \sim O(R_{\max}^2/(1 - \gamma)^2)$$

高方差的来源:

1. $(1 - \gamma)$ 小 (长视野) $\rightarrow$ 方差大
2. $R_{\max}$ 大 (奖励范围大) $\rightarrow$ 方差大
3. $\|\nabla \log \pi\|$ 大 (策略尖锐) $\rightarrow$ 方差大

方差缩减策略:

1. 用 baseline 减小 $\hat{A}$ 的绝对值
2. 奖励归一化
3. 多样本平均

Actor-Critic = 双层优化

Actor-Critic 可以看作一个**双层优化**问题：

外层 (Actor): $\max_{\theta} J(\theta) = \mathbb{E}_s[V^{\pi_{\theta}}(s)]$

内层 (Critic): $\min_{\phi} \mathbb{E}[(V_{\phi}(s) - V^{\pi_{\theta}}(s))^2]$

交替优化:

1. 固定 θ , 更新 ϕ 使 $V_{\phi} \approx V^{\pi_{\theta}}$ (策略评估)
2. 固定 ϕ , 更新 θ 改进策略 (策略改进)

这就是**广义策略迭代 (GPI)** 的神经网络版本!

定理 5.4 (策略迭代的单调性). 设 $\pi_{k+1}(s) = \arg \max_a Q^{\pi_k}(s, a)$ (贪婪策略), 则 $V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ 对所有 s 成立。

证明. Step 1: 贪婪改进的性质

由 π_{k+1} 的定义:

$$Q^{\pi_k}(s, \pi_{k+1}(s)) = \max_a Q^{\pi_k}(s, a) \geq Q^{\pi_k}(s, \pi_k(s)) = V^{\pi_k}(s)$$

Step 2: 迭代不等式

$$\begin{aligned} V^{\pi_k}(s) &\leq Q^{\pi_k}(s, \pi_{k+1}(s)) \\ &= R(s, \pi_{k+1}(s)) + \gamma \mathbb{E}_{s'}[V^{\pi_k}(s')] \\ &\leq R(s, \pi_{k+1}(s)) + \gamma \mathbb{E}_{s'}[Q^{\pi_k}(s', \pi_{k+1}(s'))] \\ &= R(s, \pi_{k+1}(s)) + \gamma \mathbb{E}_{s'}[R(s', \pi_{k+1}(s')) + \gamma \mathbb{E}_{s''}[V^{\pi_k}(s'')]] \\ &\leq \dots \end{aligned}$$

Step 3: 展开到无穷

$$V^{\pi_k}(s) \leq \mathbb{E}_{\pi_{k+1}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi_{k+1}(s_t)) \mid s_0 = s \right] = V^{\pi_{k+1}}(s)$$

□

例 5.2 (Actor-Critic——实时竞价). **场景:** 广告实时竞价 (RTB), 每秒处理 10000 次竞价请求
为什么需要 Actor-Critic:

- REINFORCE: 要等一整天的广告展示效果才能更新 → 太慢
- DQN: $\arg \max$ 在连续出价空间上难算
- Actor-Critic: 每次竞价后立即更新, 支持连续出价

Actor: π_θ (出价|用户特征), 输出出价金额
Critic: V_ϕ (状态), 估计当前预算状态的长期价值
TD 误差: $\delta = \text{广告收益} + \gamma V_\phi(\text{新预算状态}) - V_\phi(\text{旧预算状态})$
实时更新: 每次竞价后, 用 δ 更新 Actor 和 Critic。

5.3 On-policy vs Off-policy: A2C 与 Q-learning 的分野

一句话区分

On-policy (同策略): 用来更新的数据, 必须由当前正在优化的那个策略采出来; 策略一变, 旧数据就作废。

Off-policy (异策略): 可以用别的策略 (旧策略、探索策略、甚至别的智能体) 产生的数据, 来学习目标策略。

A2C 是 on-policy 的典型, Q-learning (及 DQN) 是 off-policy 的典型。两者的差别不是实现细节, 而是从学什么就分岔了。

为什么 A2C 是 on-policy? 策略梯度的期望 $\nabla_\theta J = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta}[\dots]$ 是在当前 π_θ 下取的; TD 误差 $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ 估的也是当前策略的 V^{π_θ} 。一旦 θ 更新, 状态分布 d^π 和动作分布 π_θ 都变了, 之前采的样本不再服从新分布——**必须丢弃**, 不能塞进 replay buffer 反复用 (除非做重要性采样修正, 见下)。

为什么 Q-learning 是 off-policy? 它学的是最优动作值 Q^* , 更新目标来自 Bellman 最优方程里的 $\max$:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

这个 $\max$ 对应的是贪婪的目标策略, 与产生 (s, a, r, s') 的行为策略无关。所以行为策略可以是 ϵ -greedy、可以是旧网络、可以是 replay buffer 里任意时刻存下的数据——目标值都不变。这正是 DQN 能用经验回放的根本原因。

维度	A2C (on-policy)	Q-learning / DQN (off-policy)
学什么	策略 π_θ (+ critic V^π)	最优动作值 Q^*
更新目标	$r + \gamma V^\pi(s')$ (当前策略)	$r + \gamma \max_{a'} Q(s', a')$ (贪婪)
数据来源	必须来自当前 π_θ	任意行为策略
Replay buffer	不能直接用	可以, 且是核心
样本效率	低 (用完即弃)	高 (数据可复用)
稳定性	较稳	deadly triad, 易发散
探索	靠 π_θ 自身随机性	行为策略独立 (如 ϵ -greedy)
连续动作	天然支持	$\max$ 难算, 需 DDPG/SAC

数学根源 + 通往 TRPO 的桥

on/off 的本质是期望在谁的分布下取。策略梯度要求 $a \sim \pi_\theta$; 若样本来自旧策略 π_{old} , 就

有分布失配，需乘**重要性采样比** $\rho = \pi_\theta(a|s)/\pi_{\text{old}}(a|s)$ 来纠偏：

$$\nabla_\theta J \approx \mathbb{E}_{a \sim \pi_{\text{old}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)} \nabla_\theta \log \pi_\theta(a|s) A^{\pi_{\text{old}}}(s, a) \right]$$

这正是把 on-policy 方法”半 off-policy 化”、从而能拿一小批旧数据多更新几步的关键——也是后面 TRPO 代理目标的出发点（用的就是同一个比率 ρ ）。一句话：A2C 严格 on-policy；用重要性比把它放宽到”近端（proximal）”地复用旧数据，在样本效率和稳定性之间取折中。

deadly triad: off-policy 不是免费的

off-policy 能复用数据，代价是稳定性。当**函数逼近 + 自举（bootstrapping，用估计更新估计） + off-policy** 三者同时出现时，值函数可能发散——这就是 Sutton 所说的”致命三要素”。DQN 靠 target network、replay buffer 等工程手段缓解。A2C 作为 on-policy 方法避开了第三项，所以训练通常更稳，但要为此付出”样本用完即弃”的代价。

Algorithm 3 Advantage Actor-Critic (A2C)

- 1: 初始化 Actor π_θ 和 Critic V_ϕ
 - 2: **for** 每步 t **do**
 - 3: 采样动作 $a_t \sim \pi_\theta(\cdot|s_t)$
 - 4: 执行 a_t ，观察 r_t, s_{t+1}
 - 5: 计算 TD 误差: $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
 - 6: Critic 更新: $\phi \leftarrow \phi + \beta \delta_t \nabla_\phi V_\phi(s_t)$
 - 7: Actor 更新: $\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \log \pi_\theta(a_t|s_t)$
 - 8: **end for**
-

6 Fisher 信息与自然梯度

普通梯度的问题

普通梯度下降: $\theta \leftarrow \theta + \alpha \nabla_\theta J$

问题: 参数空间中的小步，可能是策略空间中的**大步**！

例子: Softmax 策略 $\pi_\theta(a) = \frac{e^{\theta_a}}{\sum_b e^{\theta_b}}$

如果 $\theta_1 = 10, \theta_2 = 0$ ，则 $\pi(a_1) \approx 0.9999$

θ_1 从 10 变到 11（参数小变化）， $\pi(a_1)$ 几乎不变。

但 θ_2 从 0 变到 10（同样的参数变化）， $\pi(a_2)$ 从 0.0001 变到 0.5（策略巨变）！

问题: 普通梯度不知道这种差异，可能导致策略剧烈变化，训练不稳定。

6.1 Fisher 信息矩阵

定义 6.1 (Fisher 信息矩阵).

$$F_\theta = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^T]$$

F_θ 是 $d \times d$ 的正定矩阵, 衡量参数变化对分布的影响。

Fisher 信息的几何意义

F_θ 度量了策略分布 π_θ 对参数 θ 的敏感度。

KL 散度的二阶近似:

$$\text{KL}(\pi_\theta \| \pi_{\theta+\Delta\theta}) \approx \frac{1}{2} \Delta\theta^T F_\theta \Delta\theta$$

意思: 参数变化 $\Delta\theta$ 导致的策略变化 (用 KL 度量) 由 F_θ 决定。

F_θ 的特征值大 $\rightarrow$ 沿该方向参数小变化导致策略大变化

F_θ 的特征值小 $\rightarrow$ 沿该方向参数大变化也只导致策略小变化

KL 散度二阶展开的推导. 设 $p_\theta = \pi_\theta(a|s)$, $q = \pi_{\theta+\Delta\theta}(a|s)$ 。

$$\begin{aligned} \text{KL}(p||q) &= \sum_a p_\theta(a|s) \log \frac{p_\theta(a|s)}{q(a|s)} \\ &= \sum_a p_\theta(a|s) [\log p_\theta(a|s) - \log \pi_{\theta+\Delta\theta}(a|s)] \end{aligned}$$

对 $\log \pi_{\theta+\Delta\theta}$ 在 θ 处 Taylor 展开:

$$\log \pi_{\theta+\Delta\theta} \approx \log \pi_\theta + \Delta\theta^T \nabla \log \pi_\theta + \frac{1}{2} \Delta\theta^T \nabla^2 \log \pi_\theta \Delta\theta$$

代入:

$$\begin{aligned} \text{KL} &\approx \sum_a p_\theta \left[-\Delta\theta^T \nabla \log \pi_\theta - \frac{1}{2} \Delta\theta^T \nabla^2 \log \pi_\theta \Delta\theta \right] \\ &= -\Delta\theta^T \underbrace{\mathbb{E}_a[\nabla \log \pi]}_{=0} - \frac{1}{2} \Delta\theta^T \mathbb{E}_a[\nabla^2 \log \pi] \Delta\theta \end{aligned}$$

第一项为零因为 $\mathbb{E}_a[\nabla \log \pi] = \sum_a \pi \frac{\nabla \pi}{\pi} = \nabla \sum_a \pi = \nabla 1 = 0$ 。

关键引理: $\mathbb{E}[\nabla^2 \log \pi] = -\mathbb{E}[\nabla \log \pi (\nabla \log \pi)^T] = -F$

证明: $\nabla^2 \log \pi = \nabla(\nabla \log \pi) = \nabla \left(\frac{\nabla \pi}{\pi} \right) = \frac{\nabla^2 \pi}{\pi} - \frac{\nabla \pi (\nabla \pi)^T}{\pi^2}$
 $\mathbb{E}[\nabla^2 \log \pi] = \underbrace{\sum_a \nabla^2 \pi}_{=\nabla^2 1=0} - \sum_a \frac{(\nabla \pi)(\nabla \pi)^T}{\pi} = -\mathbb{E} \left[\frac{\nabla \pi}{\pi} \frac{(\nabla \pi)^T}{\pi} \right] = -F$

因此:

$$\text{KL}(p||q) \approx \frac{1}{2} \Delta\theta^T F_\theta \Delta\theta$$

□

高斯策略的 Fisher 矩阵

设置：一维高斯 $\pi_\theta(a|s) = \mathcal{N}(\mu, \sigma^2)$, $\mu = \theta$ (标量)

计算 $\nabla_\theta \log \pi$:

$$\begin{aligned}\log \pi &= -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(a - \theta)^2}{2\sigma^2} \\ \nabla_\theta \log \pi &= \frac{a - \theta}{\sigma^2}\end{aligned}$$

计算 Fisher:

$$\begin{aligned}F_\theta &= \mathbb{E}_{a \sim \mathcal{N}(\theta, \sigma^2)} \left[\left(\frac{a - \theta}{\sigma^2} \right)^2 \right] \\ &= \frac{1}{\sigma^4} \mathbb{E}[(a - \theta)^2] \\ &= \frac{1}{\sigma^4} \cdot \sigma^2 = \frac{1}{\sigma^2}\end{aligned}$$

直觉:

- σ 小 (确定性高) $\rightarrow F$ 大 $\rightarrow$ 参数小变化导致分布大变化
- σ 大 (不确定性高) $\rightarrow F$ 小 $\rightarrow$ 参数变化对分布影响小

数值例子:

- $\sigma = 0.1$: $F = 100$, θ 变 0.1 会显著改变分布
- $\sigma = 1$: $F = 1$, θ 变 0.1 影响较小
- $\sigma = 10$: $F = 0.01$, θ 变 0.1 几乎无影响

多维高斯的 Fisher 矩阵

设置: $a \in \mathbb{R}^m$, $\pi(a|s) = \mathcal{N}(\mu_\theta, \Sigma)$, Σ 固定

参数: $\mu_\theta = W\phi(s)$, $W \in \mathbb{R}^{m \times d}$, 参数是 $\text{vec}(W)$

Score:

$$\nabla_W \log \pi = \Sigma^{-1}(a - \mu)\phi(s)^T$$

Fisher 矩阵 (对 $\text{vec}(W)$):

$$F = \mathbb{E}[\text{vec}(\Sigma^{-1}(a - \mu)\phi^T) \text{vec}(\Sigma^{-1}(a - \mu)\phi^T)^T]$$

利用 $\text{vec}(ABC) = (C^T \otimes A)\text{vec}(B)$ 和 $\mathbb{E}[(a - \mu)(a - \mu)^T] = \Sigma$:

$$F = (\mathbb{E}[\phi\phi^T]) \otimes \Sigma^{-1}$$

这是 Kronecker 积形式! K-FAC 算法就是利用这种结构近似神经网络的 Fisher。

Softmax 策略的 Fisher 矩阵

设置：\$K\$ 个动作，\$\pi_\theta(a|s) = \text{softmax}(\theta^T \phi(s))_a\$

为简化，设 \$\phi(s) = 1\$ (忽略状态)，则 \$\pi_a = \frac{e^{\theta_a}}{\sum_b e^{\theta_b}}\$

Score: \$\nabla_{\theta_a} \log \pi_a = 1 - \pi_a\$, \$\nabla_{\theta_b} \log \pi_a = -\pi_b\$ (\$b \neq a\$)

写成向量：\$\nabla_{\theta} \log \pi_a = e_a - \pi\$, 其中 \$e_a\$ 是第 \$a\$ 个标准基向量。

Fisher 矩阵:

$$\begin{aligned} F &= \mathbb{E}_{a \sim \pi} [(e_a - \pi)(e_a - \pi)^T] \\ &= \sum_a \pi_a (e_a - \pi)(e_a - \pi)^T \\ &= \sum_a \pi_a [e_a e_a^T - e_a \pi^T - \pi e_a^T + \pi \pi^T] \\ &= \text{diag}(\pi) - \pi \pi^T \end{aligned}$$

验证: \$K = 2\$, \$\pi = (p, 1 - p)\$

$$F = \begin{pmatrix} p & 0 \\ 0 & 1 - p \end{pmatrix} - \begin{pmatrix} p^2 & p(1 - p) \\ p(1 - p) & (1 - p)^2 \end{pmatrix} = \begin{pmatrix} p(1 - p) & -p(1 - p) \\ -p(1 - p) & p(1 - p) \end{pmatrix}$$

\$\det(F) = 0\$! \$F\$ 是奇异的，因为 \$\sum_a \pi_a = 1\$ 约束导致参数冗余。

数值例子: \$K = 3\$, \$\pi = (0.5, 0.3, 0.2)\$

$$F = \begin{pmatrix} 0.5 & 0 & 0 \\ 0 & 0.3 & 0 \\ 0 & 0 & 0.2 \end{pmatrix} - \begin{pmatrix} 0.25 & 0.15 & 0.1 \\ 0.15 & 0.09 & 0.06 \\ 0.1 & 0.06 & 0.04 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.15 & -0.1 \\ -0.15 & 0.21 & -0.06 \\ -0.1 & -0.06 & 0.16 \end{pmatrix}$$

对角元素：各动作概率变化的敏感度。非对角元素：动作之间的耦合（负相关，因为概率和为 1）。

例 6.1 (Fisher 信息的直觉——地形图)。想象你在山上，要走到山顶。

普通梯度: 只看坡度 (\$\nabla J\$)，朝最陡的方向走固定步长。

问题: 有些方向地面松软（走一步实际移动很远），有些方向地面坚硬（走一步实际没动多少）。固定步长会导致：松软方向走太远，坚硬方向走太慢。

Fisher 信息 \$F\$ 就是地形硬度图: 告诉你每个方向地面有多硬。

自然梯度: 根据地形调整步长，松软方向小步，坚硬方向大步。

6.2 自然梯度

定义 6.2 (自然梯度)。

$$\tilde{\nabla}_{\theta} J = F_{\theta}^{-1} \nabla_{\theta} J$$

自然梯度 = Fisher 逆 \$\times\$ 普通梯度

定理 6.1 (自然梯度是 KL 约束下的最速上升方向)。

$$\tilde{\nabla}_{\theta} J = \arg \max_{\Delta\theta: \text{KL}(\pi_{\theta} \parallel \pi_{\theta+\Delta\theta}) \leq \epsilon} J(\theta + \Delta\theta)$$

(在一阶近似意义下)

证明. **Step 1:** 写出优化问题

目标函数一阶近似: $J(\theta + \Delta\theta) \approx J(\theta) + \nabla_{\theta} J^T \Delta\theta$

KL 约束二阶近似: $\text{KL}(\pi_{\theta} \parallel \pi_{\theta+\Delta\theta}) \approx \frac{1}{2} \Delta\theta^T F_{\theta} \Delta\theta$

优化问题:

$$\max_{\Delta\theta} \nabla_{\theta} J^T \Delta\theta \quad \text{s.t.} \quad \frac{1}{2} \Delta\theta^T F_{\theta} \Delta\theta \leq \epsilon$$

Step 2: 构造 Lagrangian

$$\mathcal{L}(\Delta\theta, \lambda) = \nabla_{\theta} J^T \Delta\theta - \lambda \left(\frac{1}{2} \Delta\theta^T F_{\theta} \Delta\theta - \epsilon \right)$$

Step 3: KKT 条件

对 $\Delta\theta$ 求导:

$$\nabla_{\Delta\theta} \mathcal{L} = \nabla_{\theta} J - \lambda F_{\theta} \Delta\theta = 0$$

解得: $\Delta\theta = \frac{1}{\lambda} F_{\theta}^{-1} \nabla_{\theta} J$

Step 4: 确定 λ

约束取等 (最大化时约束必紧):

$$\frac{1}{2} \cdot \frac{1}{\lambda^2} (F^{-1} \nabla J)^T F (F^{-1} \nabla J) = \epsilon$$

$$\frac{1}{2\lambda^2} (\nabla J)^T F^{-1} \nabla J = \epsilon$$

$$\lambda = \sqrt{\frac{(\nabla J)^T F^{-1} \nabla J}{2\epsilon}}$$

Step 5: 最优方向

$$\Delta\theta^* = \frac{1}{\lambda} F^{-1} \nabla J = \sqrt{\frac{2\epsilon}{(\nabla J)^T F^{-1} \nabla J}} F^{-1} \nabla J$$

方向是 $F^{-1} \nabla J$, 步长由 ϵ 决定。

因此自然梯度 $\tilde{\nabla} J = F^{-1} \nabla J$ 是 KL 约束下的最速上升方向。 $\square$

定理 6.2 (自然梯度的参数化不变性). 自然梯度对参数化方式不变: 若 $\theta' = h(\theta)$ 是参数变换, 则在新参数下的自然梯度等价于原参数下的。

证明. **Step 1:** 参数变换下的梯度变换

设 $\theta' = h(\theta)$, J 关于 θ' 的梯度:

$$\nabla_{\theta'} J = \left(\frac{\partial \theta}{\partial \theta'} \right)^T \nabla_{\theta} J = (H^{-1})^T \nabla_{\theta} J$$

其中 $H = \frac{\partial \theta'}{\partial \theta}$ 是 Jacobian。

Step 2: 参数变换下的 Fisher 变换

$$F_{\theta'} = \mathbb{E}[\nabla_{\theta'} \log \pi (\nabla_{\theta'} \log \pi)^T]$$

$$\nabla_{\theta'} \log \pi = (H^{-1})^T \nabla_{\theta} \log \pi$$

$$F_{\theta'} = (H^{-1})^T F_{\theta} H^{-1}$$

Step 3: 自然梯度的变换

$$\tilde{\nabla}_{\theta'} J = F_{\theta'}^{-1} \nabla_{\theta'} J = [H F_{\theta}^{-1} H^T] [(H^{-1})^T \nabla_{\theta} J] = H F_{\theta}^{-1} \nabla_{\theta} J = H \tilde{\nabla}_{\theta} J$$

Step 4: 验证等价性

在 θ 参数下走 $\tilde{\nabla}_{\theta} J$ 步，对应在 θ' 参数下走 $H \tilde{\nabla}_{\theta} J = \tilde{\nabla}_{\theta'} J$ 步。

这正是自然梯度在新参数下的定义！

对比普通梯度： $\nabla_{\theta'} J = (H^{-1})^T \nabla_{\theta} J \neq H \nabla_{\theta} J$

普通梯度依赖于参数化方式，自然梯度不依赖。 □

例 6.2 (参数化不变性的意义). 考虑高斯策略 $\pi = \mathcal{N}(\mu, \sigma^2)$ 。

参数化 1: $\theta = (\mu, \sigma)$

参数化 2: $\theta' = (\mu, \log \sigma)$ (更常用, 因为 $\sigma > 0$)

普通梯度: 在两种参数化下不同, 学习动态不同。

自然梯度: 在两种参数化下等价, 学习动态相同。

这意味着自然梯度优化的是分布本身, 而不是某个特定的参数化。

定理 6.3 (Fisher 信息与 KL 散度的 Hessian).

$$F_{\theta} = \nabla_{\theta'}^2 \text{KL}(\pi_{\theta} \| \pi_{\theta'}) \big|_{\theta'=\theta}$$

Fisher 矩阵是 KL 散度关于第二个参数的 Hessian。

证明. Step 1: 写出 KL 散度

$$\text{KL}(\pi_{\theta} \| \pi_{\theta'}) = \mathbb{E}_{a \sim \pi_{\theta}} [\log \pi_{\theta}(a) - \log \pi_{\theta'}(a)]$$

Step 2: 对 θ' 求一阶导

$$\nabla_{\theta'} \text{KL} = -\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta'} \log \pi_{\theta'}(a)]$$

在 $\theta' = \theta$ 处:

$$\nabla_{\theta'} \text{KL} \big|_{\theta'=\theta} = -\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a)] = 0$$

(score 的期望为零)

Step 3: 对 θ' 求二阶导

$$\nabla_{\theta'}^2 \text{KL} = -\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta'}^2 \log \pi_{\theta'}(a)]$$

在 $\theta' = \theta$ 处:

$$\nabla_{\theta'}^2 \text{KL} \big|_{\theta'=\theta} = -\mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta}^2 \log \pi_{\theta}(a)]$$

Step 4: 利用恒等式

$$\nabla^2 \log \pi = \frac{\nabla^2 \pi}{\pi} - \frac{\nabla \pi (\nabla \pi)^T}{\pi^2} = \frac{\nabla^2 \pi}{\pi} - (\nabla \log \pi)(\nabla \log \pi)^T$$

取期望：

$$\begin{aligned}\mathbb{E}[\nabla^2 \log \pi] &= \mathbb{E}\left[\frac{\nabla^2 \pi}{\pi}\right] - \mathbb{E}[(\nabla \log \pi)(\nabla \log \pi)^T] \\ &= \sum_a \nabla^2 \pi(a) - F \\ &= \nabla^2 \sum_a \pi(a) - F \\ &= 0 - F = -F\end{aligned}$$

因此：

$$\nabla_{\theta'}^2 \text{KL}|_{\theta'=\theta} = -(-F) = F$$

□

自然梯度与预条件优化

回忆 Week 3 学的**预条件梯度下降**：

$$\theta \leftarrow \theta - \alpha P^{-1} \nabla L$$

其中 P 是预条件矩阵，好的 P 能加速收敛。

自然梯度就是用 Fisher 矩阵作为预条件！

Newton 法： $P = \nabla^2 L$ (Hessian) $\rightarrow$ 考虑目标函数的曲率

自然梯度： $P = F$ (Fisher) $\rightarrow$ 考虑**参数化**的曲率

区别： Newton 关心”目标函数的形状”，自然梯度关心”策略空间的形状”。

联系： 对于负对数似然损失， $F = \mathbb{E}[\nabla^2(-\log p)] = \mathbb{E}[\text{Hessian}]$ ，Fisher 等于期望 Hessian！

自然梯度的计算复杂度

F_θ 是 $d \times d$ 矩阵， d 是参数量（神经网络通常 $10^6 \sim 10^9$ ）。

直接计算 F^{-1} ： $O(d^3) \rightarrow$ 不可行！

解决方案：

1. **共轭梯度**：不显式求 F^{-1} ，只需要 Fv 的矩阵-向量乘积
2. **Kronecker 近似** (K-FAC)：利用网络结构近似 F
3. **对角近似**：只保留 F 的对角元素

7 TRPO 的理论基础：策略改进界

TRPO 的核心问题

我们想改进策略 $\pi_\theta \rightarrow \pi_{\theta'}$ ，但：

1. 如何保证新策略确实更好？
2. 如何用旧策略的数据评估新策略？

Kakade & Langford (2002) 给出了漂亮的理论回答。

7.1 Performance Difference Lemma

定理 7.1 (Performance Difference Lemma). 对任意两个策略 π 和 π' ：

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}, a \sim \pi'} [A^\pi(s, a)]$$

其中 $d^{\pi'}$ 是 π' 的状态访问分布， $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 是 π 的优势函数。

证明. **Step 1:** 定义并展开 $V^{\pi'}(s)$

$$V^{\pi'}(s_0) = \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 \right]$$

Step 2: 引入 V^π 进行配对

关键技巧：加减 $V^\pi(s_t)$

$$\begin{aligned} V^{\pi'}(s_0) &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \\ &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + V^\pi(s_t) - V^\pi(s_t)) \right] \\ &= \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)) + \sum_{t=0}^{\infty} \gamma^t (V^\pi(s_t) - \gamma V^\pi(s_{t+1})) \right] \end{aligned}$$

Step 3: 处理第二项 (telescoping)

$$\begin{aligned} \sum_{t=0}^{\infty} \gamma^t (V^\pi(s_t) - \gamma V^\pi(s_{t+1})) &= V^\pi(s_0) + \sum_{t=1}^{\infty} \gamma^t V^\pi(s_t) - \sum_{t=0}^{\infty} \gamma^{t+1} V^\pi(s_{t+1}) \\ &= V^\pi(s_0) + \sum_{t=1}^{\infty} \gamma^t V^\pi(s_t) - \sum_{t=1}^{\infty} \gamma^t V^\pi(s_t) \\ &= V^\pi(s_0) \end{aligned}$$

Step 4: 识别优势函数

第一项中：

$$r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)$$

因为 $Q^\pi(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[V^\pi(s')]$, 在给定 (s_t, a_t, s_{t+1}) 的条件下, $r_t + \gamma V^\pi(s_{t+1})$ 是 $Q^\pi(s_t, a_t)$ 的无偏估计。

Step 5: 整合

$$V^{\pi'}(s_0) = V^\pi(s_0) + \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t A^\pi(s_t, a_t) \right]$$

对初始状态分布取期望:

$$\begin{aligned} J(\pi') &= J(\pi) + \mathbb{E}_{s_0 \sim \rho_0} \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t A^\pi(s_t, a_t) \right] \\ &= J(\pi) + \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim d_t^{\pi'}, a_t \sim \pi'} [A^\pi(s_t, a_t)] \end{aligned}$$

其中 $d_t^{\pi'}(s)$ 是 t 步时的状态分布。

Step 6: 用状态访问分布整理

定义折扣状态访问分布: $d^{\pi'}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t d_t^{\pi'}(s)$

则:

$$J(\pi') - J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi'}, a \sim \pi'} [A^\pi(s, a)]$$

□

推论 7.1 (策略改进条件). 如果对所有状态 s , $\mathbb{E}_{a \sim \pi'} [A^\pi(s, a)] \geq 0$, 则 $J(\pi') \geq J(\pi)$ 。

证明. 直接由 Performance Difference Lemma:

$$J(\pi') - J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi'}} [\mathbb{E}_{a \sim \pi'} [A^\pi(s, a)]] \geq 0$$

□

7.2 重要性采样: 用旧策略的数据估计新策略的期望

要解决的矛盾

Performance Difference Lemma 里要算 $\mathbb{E}_{a \sim \pi'} [A^\pi]$ ——期望在新策略 π' 下取; 可我们手里只有旧策略 π 采的样本。重要性采样就是搭这座桥的工具。

引理 7.1 (重要性采样). 对任意函数 f 与两个策略 π, π' (要求 $\pi(a|s) > 0$ 处才有 $\pi'(a|s)f \neq 0$),

$$\mathbb{E}_{a \sim \pi'} [f(a)] = \mathbb{E}_{a \sim \pi} \left[\frac{\pi'(a|s)}{\pi(a|s)} f(a) \right].$$

证明. 换测度, 一行即得:

$$\mathbb{E}_{a \sim \pi'} [f(a)] = \sum_a \pi'(a|s) f(a) = \sum_a \pi(a|s) \frac{\pi'(a|s)}{\pi(a|s)} f(a) = \mathbb{E}_{a \sim \pi} \left[\frac{\pi'(a|s)}{\pi(a|s)} f(a) \right].$$

记**重要性比率** $\rho(a|s) = \frac{\pi'(a|s)}{\pi(a|s)}$: 旧策略不爱选、新策略偏爱的动作, $\rho > 1$ 被放大; 反之被压低; $\pi' = \pi$ 时 $\rho \equiv 1$ 。(要求 π 在 π' 有质量处也有质量, 否则比率没定义——即”提议分布要覆盖目标的支撑”。)

□

用到优势上：取 $f = A^\pi(s, \cdot)$,

$$\mathbb{E}_{a \sim \pi'}[A^\pi(s, a)] = \mathbb{E}_{a \sim \pi}[\rho(a|s) A^\pi(s, a)].$$

于是下一节的代理目标里那项 $\mathbb{E}_{s \sim d^\pi, a \sim \pi'}[A^\pi]$ ，就能完全用旧策略 π 采的数据估计：

$$\mathbb{E}_{s \sim d^\pi, a \sim \pi}[\rho(a|s) A^\pi(s, a)].$$

这正是 TRPO 接下来要最大化的代理目标。

两个代价，正好引出信任域

(1) 只换了动作分布，没换状态分布。严格的 $J(\pi')$ 用的是新状态访问分布 $d^{\pi'}$ ，可我们只能用旧的 d^π ；代理目标默认 $d^{\pi'} \approx d^\pi$ ——这是近似，仅当 $\pi' \approx \pi$ 时才准。

(2) 比率方差随分布失配暴涨。重要性采样无偏，但 π' 离 π 越远， ρ 越可能极端（趋于 0 或很大），估计方差越大。

两个代价都指向同一个药方：把 π' 锁在 π 附近。这就是 TRPO 要约束 $\bar{D}_{KL}(\pi, \pi') \leq \delta$ 的根本动机——不是“为保守而保守”，而是为了让重要性采样的代理目标可信。

注记 7.1 (为什么不用整条轨迹的 IS). 若对整条轨迹做重要性采样，权重是每步比率的连乘 $\prod_t \rho(a_t|s_t)$ ，方差随轨迹长度**指数爆炸**，实践不可用。这里用的是逐状态的单步动作比率（状态仍按 d^π 采），把连乘降成单个 $\rho(a|s)$ 。

7.3 代理目标与近似界

问题

Performance Difference Lemma 需要在 π' 的分布下采样，但我们只有 π 的数据！

解决：定义一个可以用 π 数据计算的**代理目标**，并证明它和真实目标的差距有界。

定义 7.1 (代理目标).

$$L_\pi(\pi') = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^\pi, a \sim \pi'}[A^\pi(s, a)]$$

注意：状态分布是 d^π （旧策略），但动作分布是 π' （新策略）。

引理 7.2 (代理目标在 π 处的性质). 1. $L_\pi(\pi) = J(\pi)$

$$2. \nabla_{\pi'} L_\pi(\pi')|_{\pi'=\pi} = \nabla_{\pi'} J(\pi')|_{\pi'=\pi}$$

即代理目标是真实目标的一阶近似。

证明. 性质 1:

$$L_\pi(\pi) = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^\pi, a \sim \pi}[A^\pi(s, a)] = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_s[\underbrace{\mathbb{E}_{a \sim \pi}[A^\pi(s, a)]}_{=0}] = J(\pi)$$

性质 2: 由 Performance Difference Lemma，真实目标可精确写成

$$J(\pi') = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}}[h(s, \pi')], \quad h(s, \pi') \triangleq \mathbb{E}_{a \sim \pi'}[A^\pi(s, a)].$$

它与代理目标 $L_\pi(\pi')$ 的唯一区别在状态分布： J 用随 π' 变化的 $d^{\pi'}$ ， L_π 把它固定成 d^π 。对 π' 求梯度时， J 这边的状态分布也要被求导，按乘积法则分成两项：

$$\nabla_{\pi'} J(\pi') = \frac{1}{1-\gamma} \underbrace{\sum_s (\nabla_{\pi'} d^{\pi'}(s)) h(s, \pi')}_{\text{(I) 状态分布变化项}} + \frac{1}{1-\gamma} \underbrace{\sum_s d^{\pi'}(s) \nabla_{\pi'} h(s, \pi')}_{\text{(II) 动作分布变化项}}.$$

在 $\pi' = \pi$ 处分别看：

(I) 恰为零——不是因为 $\nabla_{\pi'} d^{\pi'} = 0$ （它一般不为零），而是因为它的系数 $h(s, \pi) = \mathbb{E}_{a \sim \pi}[A^\pi(s, a)] = 0$ （优势在 π 下均值为零）。系数为零，整项就被乘没了。

(II) 正好等于 $\nabla_{\pi'} L_\pi$ ——代理目标把状态分布钉死在 d^π ，故

$$\nabla_{\pi'} L_\pi(\pi')|_{\pi'=\pi} = \frac{1}{1-\gamma} \sum_s d^\pi(s) \nabla_{\pi'} h(s, \pi')|_{\pi'=\pi},$$

而 $\pi' = \pi$ 处 $d^{\pi'} = d^\pi$ ，这恰是 (II)。

合起来 $\nabla_{\pi'} J|_{\pi'=\pi} = \frac{1}{1-\gamma}[(I) + (II)] = \frac{1}{1-\gamma}(II) = \nabla_{\pi'} L_\pi|_{\pi'=\pi}$ 。

要点：代理目标”扔掉”的正是状态分布变化项 (I)，而它在 $\pi' = \pi$ 处因 $\mathbb{E}_\pi[A^\pi] = 0$ 而消失——这就是为什么 L_π 与 J 在 π 处**一阶吻合**；离开 π 后误差才出现，由下面的 Kakade-Langford 界控制。 $\square$

注记 7.2 (代理目标就是”重要性采样版”的目标)。把上一节的重要性采样 ($f = A^\pi$ ，比率 $\rho(a|s) = \pi'(a|s)/\pi(a|s)$) 用到代理目标的动作期望上：

$$L_\pi(\pi') = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^\pi, a \sim \pi'}[A^\pi] = J(\pi) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^\pi, a \sim \pi}[\rho(a|s) A^\pi(s, a)].$$

右边状态和动作都从旧策略 π 采，所以能**完全用旧数据**估计——这才是代理目标”可计算”的真正来历。把这一章串起来：Performance Difference Lemma 给出精确但需 π' 采样的差值 $\rightarrow$ 固定状态分布 ($d^{\pi'} \approx d^\pi$) 得到代理目标 $\rightarrow$ 重要性采样 ρ 再把动作期望也搬到 π 上 $\rightarrow$ 最终落到一个**纯旧数据**就能优化的目标 $\frac{1}{1-\gamma} \mathbb{E}_{d^\pi, \pi}[\rho A^\pi]$ 。TRPO 就是在” π' 离 π 不太远”的信任域里最大化它。

定理 7.2 (Kakade-Langford 界). 设 $\epsilon = \max_s |\mathbb{E}_{a \sim \pi'}[A^\pi(s, a)]|$, $D_{TV}^{\max} = \max_s D_{TV}(\pi(\cdot|s) \| \pi'(\cdot|s))$, 则：

$$J(\pi') \geq L_\pi(\pi') - \frac{4\gamma\epsilon}{(1-\gamma)^2} (D_{TV}^{\max})^2$$

证明. Step 1: 分解真实目标与代理目标的差

$$\begin{aligned} J(\pi') - L_\pi(\pi') &= \frac{1}{1-\gamma} \left[\mathbb{E}_{s \sim d^{\pi'}, a \sim \pi'}[A^\pi] - \mathbb{E}_{s \sim d^\pi, a \sim \pi'}[A^\pi] \right] \\ &= \frac{1}{1-\gamma} \mathbb{E}_{a \sim \pi'} \left[\sum_s (d^{\pi'}(s) - d^\pi(s)) A^\pi(s, a) \right] \end{aligned}$$

Step 2: 界定状态分布的差异

关键引理： $\|d^{\pi'} - d^\pi\|_1 \leq \frac{2\gamma}{1-\gamma} D_{TV}^{\max}$

证明思路：状态分布的差异来自于每一步转移概率的差异累积。

设 d_t^π 和 $d_t^{\pi'}$ 是 t 步时的状态分布。

$d_0^{\pi'} = d_0^\pi = \rho_0$ (初始分布相同)

$$\|d_{t+1}^{\pi'} - d_{t+1}^\pi\|_1 \leq \|d_t^{\pi'} - d_t^\pi\|_1 + 2D_{TV}^{\max}$$

(每一步最多增加 $2D_{TV}^{\max}$ 的差异)

递推: $\|d_t^{\pi'} - d_t^\pi\|_1 \leq 2t \cdot D_{TV}^{\max}$

对折扣状态分布:

$$\begin{aligned} \|d^{\pi'} - d^\pi\|_1 &= (1 - \gamma) \left\| \sum_{t=0}^{\infty} \gamma^t (d_t^{\pi'} - d_t^\pi) \right\|_1 \\ &\leq (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \cdot 2t \cdot D_{TV}^{\max} \\ &= 2(1 - \gamma) D_{TV}^{\max} \sum_{t=0}^{\infty} t \gamma^t \\ &= 2(1 - \gamma) D_{TV}^{\max} \cdot \frac{\gamma}{(1 - \gamma)^2} \\ &= \frac{2\gamma}{1 - \gamma} D_{TV}^{\max} \end{aligned}$$

Step 3: 代入得到界

$$\begin{aligned} |J(\pi') - L_\pi(\pi')| &\leq \frac{1}{1 - \gamma} \cdot \|d^{\pi'} - d^\pi\|_1 \cdot \epsilon \\ &\leq \frac{1}{1 - \gamma} \cdot \frac{2\gamma}{1 - \gamma} D_{TV}^{\max} \cdot \epsilon \\ &= \frac{2\gamma\epsilon}{(1 - \gamma)^2} D_{TV}^{\max} \end{aligned}$$

Step 4: 用 KL 散度替换 TV 距离

由 Pinsker 不等式: $D_{TV}(p||q) \leq \sqrt{\frac{1}{2} \text{KL}(p||q)}$

所以 $D_{TV}^{\max} \leq \sqrt{\frac{1}{2} D_{KL}^{\max}}$

代入并整理得最终界。

(注: 精确常数取决于具体推导, 这里给出的是核心思想。)

□

界的意义

$$J(\pi') \geq L_\pi(\pi') - C \cdot D_{KL}^{\max}(\pi||\pi')$$

右边是下界: 只要新策略和旧策略足够接近 (D_{KL} 小), 代理目标 $L_\pi(\pi')$ 就是真实目标 $J(\pi')$ 的好近似。

TRPO 的策略: 最大化 $L_\pi(\pi')$, 同时约束 $D_{KL} \leq \delta$, 保证下界有效。

界的数值感受

$\gamma = 0.99$, $\epsilon = 1$ (优势函数范围), $D_{KL}^{\max} = 0.01$

$$D_{TV}^{\max} \leq \sqrt{0.005} \approx 0.07$$

惩罚项: $\frac{4 \times 0.99 \times 1}{0.01^2} \times 0.07^2 \approx \frac{4}{0.0001} \times 0.005 = 200$

如果 $L_\pi(\pi') - L_\pi(\pi) = 1$ (代理目标改进 1), 真实改进至少 $1 - 200 \times 0.01 = -1...$

这说明 KL 约束需要足够紧! 实际中用 $\delta \approx 0.001 \sim 0.01$ 。

8 TRPO: 信任域策略优化

自然梯度的问题

自然梯度保证了”在 KL 球内的最优方向”。

但步长 α 还是要手调! 太大会崩, 太小收敛慢。

TRPO 的思想: 直接限制策略变化 (KL 散度), 而不是参数变化。

8.1 TRPO 的推导

定义 8.1 (TRPO 优化问题).

$$\max_{\theta'} L_\theta(\theta') \quad \text{s.t.} \quad \mathbb{E}_s[\text{KL}(\pi_\theta(\cdot|s) \parallel \pi_{\theta'}(\cdot|s))] \leq \delta$$

这是一个信任域 (Trust Region) 问题!

TRPO 与信任域方法

回忆 Week 4 学的信任域方法:

$$\min_{\Delta x} m(\Delta x) \quad \text{s.t.} \quad \|\Delta x\| \leq \delta$$

其中 m 是目标函数的局部模型。

TRPO 完全对应:

- $m(\Delta\theta) = -L_\theta(\theta + \Delta\theta)$ (负代理目标)
- $\|\cdot\|$ 换成 KL 散度
- δ 是信任域半径

区别:

- 传统信任域用欧氏距离: $\|\Delta x\|_2 \leq \delta$
- TRPO 用 KL 散度: $\text{KL}(\pi_\theta \parallel \pi_{\theta'}) \leq \delta$

为什么用 KL? 因为 KL 度量的是策略的真实变化, 而欧氏距离只是参数的变化。

8.2 TRPO 的求解

二阶近似

将 $L_\theta(\theta')$ 一阶展开, KL 约束二阶展开:

目标: $L_\theta(\theta') \approx L_\theta(\theta) + \nabla_\theta L_\theta(\theta)^T(\theta' - \theta) = g^T \Delta\theta$

约束: $\text{KL} \approx \frac{1}{2} \Delta\theta^T F \Delta\theta$

优化问题:

$$\max_{\Delta\theta} g^T \Delta\theta \quad \text{s.t.} \quad \frac{1}{2} \Delta\theta^T F \Delta\theta \leq \delta$$

这正是自然梯度的推导!

定理 8.1 (TRPO 更新方向).

$$\Delta\theta^* = \sqrt{\frac{2\delta}{g^T F^{-1} g}} \cdot F^{-1} g$$

其中 $g = \nabla_\theta L_\theta(\theta)$ 。

证明. Step 1: 写出 Lagrangian

原问题: $\max_{\Delta\theta} g^T \Delta\theta \quad \text{s.t.} \quad \frac{1}{2} \Delta\theta^T F \Delta\theta \leq \delta$

Lagrangian: $\mathcal{L}(\Delta\theta, \lambda) = g^T \Delta\theta - \lambda \left(\frac{1}{2} \Delta\theta^T F \Delta\theta - \delta \right)$

Step 2: 对 $\Delta\theta$ 求导

$$\nabla_{\Delta\theta} \mathcal{L} = g - \lambda F \Delta\theta = 0$$

$$\text{解得: } \Delta\theta = \frac{1}{\lambda} F^{-1} g$$

Step 3: 代入约束确定 λ

约束取等号 (最大化时约束必紧):

$$\begin{aligned} \frac{1}{2} \Delta\theta^T F \Delta\theta &= \delta \\ \frac{1}{2} \cdot \frac{1}{\lambda^2} (F^{-1} g)^T F (F^{-1} g) &= \delta \\ \frac{1}{2\lambda^2} g^T F^{-1} F F^{-1} g &= \delta \\ \frac{1}{2\lambda^2} g^T F^{-1} g &= \delta \end{aligned}$$

$$\text{解得: } \lambda = \sqrt{\frac{g^T F^{-1} g}{2\delta}}$$

Step 4: 代回得到最优解

$$\Delta\theta^* = \frac{1}{\lambda} F^{-1} g = \sqrt{\frac{2\delta}{g^T F^{-1} g}} F^{-1} g$$

$$\text{验证: } \|\Delta\theta^*\|_F^2 = \frac{2\delta}{g^T F^{-1} g} \cdot g^T F^{-1} F F^{-1} g = \frac{2\delta}{g^T F^{-1} g} \cdot g^T F^{-1} g = 2\delta \quad \checkmark$$

□

TRPO 步长的数值例子

设置: $d = 2$ 维参数, $\delta = 0.01$

$$g = \nabla_\theta L = (1, 2)^T$$

$$F = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}, F^{-1} = \frac{1}{7} \begin{pmatrix} 2 & -1 \\ -1 & 4 \end{pmatrix}$$

计算 $F^{-1}g$:

$$F^{-1}g = \frac{1}{7} \begin{pmatrix} 2 & -1 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \frac{1}{7} \begin{pmatrix} 0 \\ 7 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

计算 $g^T F^{-1}g$:

$$g^T F^{-1}g = (1, 2) \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 2$$

计算步长系数:

$$\alpha = \sqrt{\frac{2\delta}{g^T F^{-1}g}} = \sqrt{\frac{0.02}{2}} = 0.1$$

最终更新:

$$\Delta\theta^* = 0.1 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0.1 \end{pmatrix}$$

对比普通梯度: $\alpha_{\text{vanilla}} \cdot g = \alpha(1, 2)^T$

即使普通梯度在 θ_1 方向有分量, TRPO 认为这个方向 Fisher 信息大 (策略敏感), 所以不走。

8.3 共轭梯度法求解 $F^{-1}g$

为什么不直接求 F^{-1}

神经网络参数 $d \sim 10^6$, F 是 $10^6 \times 10^6$ 矩阵。

存储: 10^{12} 个浮点数 $\rightarrow$ 4TB 内存

求逆: $O(d^3) = O(10^{18})$ 操作 $\rightarrow$ 不可能

解决: 我们只需要 $F^{-1}g$, 不需要 F^{-1} 本身!

用共轭梯度法 (CG): 只需要计算 Fv (矩阵-向量乘积), 不需要存储 F 。

Fisher-向量乘积的计算

$$Fv = \mathbb{E}[\nabla \log \pi (\nabla \log \pi)^T v] = \mathbb{E}[\nabla \log \pi \cdot (\nabla \log \pi)^T v]$$

$$\text{设 } \psi = \nabla_{\theta} \log \pi, \text{ 则 } Fv = \mathbb{E}[\psi \cdot \psi^T v] = \mathbb{E}[\psi \cdot (\psi \cdot v)]$$

计算步骤:

1. 对每个样本 (s_i, a_i) , 计算 $\psi_i = \nabla_{\theta} \log \pi_{\theta}(a_i | s_i)$
2. 计算标量 $c_i = \psi_i^T v$
3. 计算 $\psi_i \cdot c_i$
4. 取平均: $Fv \approx \frac{1}{N} \sum_i c_i \psi_i$

复杂度: $O(Nd)$, 其中 N 是样本数, d 是参数量。

Algorithm 4 共轭梯度法求 $F^{-1}g$

```
1: 输入: 梯度  $g$ , Fisher-向量乘积函数  $Fv$ , 迭代次数  $K$ 
2:  $x_0 = 0$ ,  $r_0 = g - Fx_0 = g$ ,  $p_0 = r_0$ 
3: for  $k = 0, 1, \dots, K - 1$  do
4:    $\alpha_k = \frac{r_k^T r_k}{p_k^T F p_k}$  ▷ 步长
5:    $x_{k+1} = x_k + \alpha_k p_k$  ▷ 更新解
6:    $r_{k+1} = r_k - \alpha_k F p_k$  ▷ 更新残差
7:    $\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$  ▷ 共轭系数
8:    $p_{k+1} = r_{k+1} + \beta_k p_k$  ▷ 更新搜索方向
9: end for
10: 返回:  $x_K \approx F^{-1}g$ 
```

CG 的收敛性

CG 在 d 步内精确收敛 (理论上), 但实际中:

- 10-20 步就足够好
- 每步 $O(Nd)$ (一次 Fv 乘积)
- 总复杂度 $O(KNd)$, $K \ll d$

数值例子: $d = 10^6$ 参数, $N = 10^4$ 样本, $K = 10$ 步

直接求逆: $O(10^{18}) \rightarrow$ 不可能

CG: $O(10 \times 10^4 \times 10^6) = O(10^{11}) \rightarrow$ 可行

加速比: 10^7 倍!

8.4 TRPO 的完整算法

Algorithm 5 TRPO 完整算法

```

1: 超参数: 信任域半径  $\delta$ , CG 迭代数  $K$ , 回溯系数  $\alpha_{\text{backtrack}}$ 
2: for iteration = 1, 2, ... do
3:   // 1. 收集数据
4:   用  $\pi_{\theta_{\text{old}}}$  收集  $N$  条轨迹
5:   计算每步的  $\hat{A}_t$  (用 GAE)
6:   // 2. 计算梯度
7:    $g = \nabla_{\theta} L(\theta)|_{\theta=\theta_{\text{old}}}$ , 其中  $L(\theta) = \mathbb{E} \left[ \frac{\pi_{\theta}}{\pi_{\theta_{\text{old}}}} \hat{A} \right]$ 
8:   // 3. 共轭梯度求  $F^{-1}g$ 
9:    $s = \text{CG}(F, g, K)$   $\triangleright s \approx F^{-1}g$ 
10:  // 4. 计算步长
11:   $\alpha = \sqrt{\frac{2\delta}{s^T F s}}$   $\triangleright$  使得  $\frac{1}{2}\Delta\theta^T F \Delta\theta = \delta$ 
12:  // 5. 线搜索 (确保改进)
13:  for  $j = 0, 1, 2, \dots$  do
14:     $\theta_{\text{new}} = \theta_{\text{old}} + \alpha_{\text{backtrack}}^j \cdot \alpha \cdot s$ 
15:    计算  $\bar{\text{KL}} = \mathbb{E}_s[\text{KL}(\pi_{\theta_{\text{old}}}(\cdot|s) \parallel \pi_{\theta_{\text{new}}}(\cdot|s))]$ 
16:    if  $\bar{\text{KL}} \leq \delta$  and  $L(\theta_{\text{new}}) \geq L(\theta_{\text{old}})$  then
17:      break
18:    end if
19:  end for
20:   $\theta_{\text{old}} \leftarrow \theta_{\text{new}}$ 
21: end for

```

例 8.1 (TRPO vs 普通策略梯度——稳定性对比). **场景**: 训练机器人行走

普通策略梯度:

- 学习率太大: 策略突变, 机器人从”能走”变成”乱摔”
- 学习率太小: 收敛太慢
- 需要仔细调学习率

TRPO:

- 直接限制策略变化: $\text{KL} \leq 0.01$
- 每次更新后, 新策略和旧策略**保证相似**
- 不会突然崩溃
- 自动调整参数空间的步长

代价: 每次更新需要计算 Fisher 矩阵 (或其近似), 计算量大。

TRPO 的理论保证

单调改进：在满足 KL 约束的情况下，TRPO 保证 $J(\theta_{\text{new}}) \geq J(\theta_{\text{old}}) - \epsilon$

其中 ϵ 取决于近似误差。

收敛性：TRPO 收敛到局部最优（在策略参数化允许的范围内）。

与信任域的联系：

	传统信任域	TRPO
变量	$x \in \mathbb{R}^d$	$\theta \in \mathbb{R}^d$
目标	$f(x)$	$J(\theta)$
约束	$\ x - x_k\ _2 \leq \delta$	$\text{KL}(\pi_\theta \ \pi_{\theta_k}) \leq \delta$
局部模型	二阶 Taylor	代理目标 L

9 算法对比与总结

算法	类型	方差	偏差	计算量
REINFORCE	Monte Carlo	高	无	低
+ Baseline	MC + 方差缩减	中	无	低
Actor-Critic	TD	中	有 (TD)	中
Natural PG	二阶方法	中	有	高
TRPO	信任域	中	有	高

优化视角的统一

REINFORCE：随机梯度上升

Baseline：方差缩减技术（控制变量法）

Actor-Critic：双层优化（内层评估，外层改进）

自然梯度：预条件优化（Fisher 预条件）

TRPO：信任域方法（KL 约束）

本讲核心信息

1. **函数逼近：**用神经网络代替表格，解决维度诅咒
2. **DQN：**Experience Replay + Target Network 实现稳定训练
3. **向量化环境：**并行仿真，加速 100-1000 倍
4. **策略梯度定理：** $\nabla J = \mathbb{E}[\nabla \log \pi \cdot Q]$ ，可用采样估计
5. **Baseline：**方差缩减， $b^* = V$ 时最优
6. **Actor-Critic：**双网络，实时更新

7. **Fisher 信息**: 度量参数变化对分布的影响
8. **自然梯度**: $F^{-1}\nabla J$, KL 空间中的最速方向
9. **TRPO**: 信任域约束, 保证策略单调改进

课后练习

1. **策略梯度**: 对于确定性策略 $\pi_\theta(s) = \theta^T \phi(s)$ (线性策略), 推导 $\nabla_\theta J$ 。
2. **Baseline**: 证明最优 baseline 为 $b^*(s) = \frac{\mathbb{E}[G^2 \|\nabla \log \pi\|^2]}{\mathbb{E}[\|\nabla \log \pi\|^2]}$ (精确最小方差)。
3. **Fisher 信息**: 对于高斯策略 $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma^2)$, 计算 Fisher 信息矩阵。
4. **TRPO**: 推导当 $\delta \rightarrow 0$ 时, TRPO 退化为自然梯度下降。
5. **运筹学应用**: 设计一个 Actor-Critic 算法用于动态车辆路径问题 (DVRP), 说明状态、动作、奖励的设计。

答案提示

1. 确定性策略梯度 (DPG): $\nabla_\theta J = \mathbb{E}[\nabla_a Q(s, a)|_{a=\pi(s)} \nabla_\theta \pi_\theta(s)]$
2. 对 b 求导令其为零, 需要考虑 $\nabla \log \pi$ 的范数。
3. 对于一维高斯, $F = 1/\sigma^2$ (参数为 μ 时)。
4. $\delta \rightarrow 0$ 时, 约束紧, TRPO 步长趋于 $\sqrt{2\delta/(g^T F^{-1} g)} F^{-1} g$, 正比于 $F^{-1} g$ 。
5. 状态: (车辆位置, 剩余容量, 待服务客户); 动作: 下一个访问客户; 奖励: -行驶距离-延迟惩罚。